POORNIMA
COLLEGE OF ENGINEERING

An autonomous institution approved by RTU, AICTE & UGC • NAAC A+ Accredited

ISI-6, RIICO Institutional Area, Sitapura, Jaipur-302022, Rajasthan
Phone/Fax: 0141-2770790-92, www.pce.poornima.org

NETWORK & SHELL PROGRAMMING LAB MANUAL

(Lab Code: 244AI4-22/244CY4-22)

4th Semester, 2nd Year

Department of Advance Computing

Session: 2025-26

TABLE OF CONTENT

	it is valid triangle or not. 4. To check whether a character is alphabet, digit or special character. 5. To calculate profit or loss.	
14	Write a shell script to print different shapes- Diamond, triangle, square, rectangle, hollow square etc.	
15	Shell Programming – Arrays.	
	Beyond the Syllabus	
1	Construct a network of hub and switch using Packet Tracer.	
2	Construct a network of Mesh topolgy using Packet Tracer.	

INSTITUTE VISION & MISSION

VISION

To create knowledge based society with scientific temper, team spirit and dignity of labor to face the global competitive challenges.

MISSION

To evolve and develop skill-based systems for effective delivery of knowledge so as to equip young professionals with dedication & commitment to excellence in all spheres of life

DEPARTMENT VISION & MISSION

VISION

Become most preferred department for the latest advanced computing programs through creating appropriate teaching-learning and skill up gradation environment that fulfill current industry needs.

MISSION

1. To create experiential learning environment that will enable students to compete globally in advanced computing domain. To contribute significantly to the research and the discovery of new.
2. To adapt latest technological tools and contribute significantly for the advancement of knowledge in computer engineering application in industry, society and environment.
3. To inculcate essential characteristic in the students for their all-round professional development, interaction with industry and society and lifelong learning.
4. To create R & D infrastructure and center of excellence in various advanced computing sub domains.

RTU SYLLABUS AND MARKING SCHEME

244AI4-22/244CY4-22: Network & Shell Programming Lab	
Credit: 1	**Max. Marks: 100 (IA:60, ETE:40)**
0L+0T+2P	**End Term Exam: 2 Hours**
S. No.	**NAME OF EXPERIMENTS**
1	Use of Basic Unix Shell Commands: ls, mkdir, rmdir, cd, cat, banner, touch, file, wc, sort, cut, grep, dd, dfspace, du, ulimit.
2	Commands related to inode, I/O redirection and piping, process control commands, mails.
3	Study of Different Type of LAN& Network Equipment.
4	Study and Verification of standard Network topologies i.e. Star, Bus, Ring etc.
5	LAN installations and Configurations.
6	Write a program to implement various types of error correcting techniques.
7	Write a program to implement various types of framing methods.
8	Write two programs in C: hello_client and hello_server a. The server listens for, and accepts, a single TCP connection; it reads all the data it can from that connection, and prints it to the screen; then it closes the connection b. The client connects to the server, sends the string “Hello, world!”, then closes the connection
9	Write an Echo_Client and Echo_server using TCP to estimate the round-trip time from client to the server. The server should be such that it can accept multiple connections at any given time.
10	Repeat Exercises 6 & 7 for UDP.
11	Repeat Exercise 7 with multiplexed I/O operations.
12	Simulate Bellman-Ford Routing algorithm in NS2.
13	Shell Programming: Shell script based on control structure- If-then-fi, if-then else-if, nested if-else, to find: 1. Greatest among three numbers. 2. To find a year is leap year or not. 3. To input angles of a triangle and find out whether it is valid triangle or not. 4. To check whether a character is alphabet, digit or special character. 5. To calculate profit or loss.
14	Write a shell script to print different shapes- Diamond, triangle, square, rectangle, hollow square etc.
15	Shell Programming – Arrays.

EVALUATION SCHEME

I+II Mid Term Examination			Attendance and performance			End Term Examination			Total Marks
Experiment	**Viva**	**Total**	**Attendance**	**Performance**	**Total**	**Experiment**	**Viva**	**Total**	
30	10	40	10	30	40	30	10	40	100

DISTRIBUTION OF MARKS FOR EACH EXPERIMENT

Attendance	Record	Performance	Total
2	3	5	10

LAB OUTCOME AND ITS MAPPING WITH PO & PSO

LAB OUTCOMES

After completion of this course, students will be able to –

243AI4-23.1	**To understand,** identify, and work with various types of local area networks (LANs), standard network equipment, and network topologies such as star, bus, and ring.
243AI4-23.2	**To develop** practical skills for LAN installation, configuration, and verification, ensuring students can design and deploy efficient and reliable network infrastructures.
243AI4-23.3	**To apply** programming knowledge for implementing error correction techniques, framing methods, and client-server socket communication using TCP and UDP protocols.
243AI4-23.4	**To provide experience** with network simulation tools (such as NS2) and Unix-based system commands, enhancing students' ability to analyze, troubleshoot, and manage network systems and processes effectively.

LO-PO-PSO MAPPING MATRIX OF COURSE

LO/PO/PSO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
243AI4-23.1	-	-	-	-	-	-	-	-	-	-	-	-	2	-	-
243AI4-23.2	-	2	-	-	-	-	-	-	-	-	-	-	2	-	-
243AI4-23.3	-	-	2	-	-	-	-	-	-	-	-	-	-	2	-
243AI4-23.4	-	-	2		-	-	-	-	-	-	-	-	2	-	2

PROGRAM OUTCOMES (POs)

PO1	**Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems
PO2	**Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3	**Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	**Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	**Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	**The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	**Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	**Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	**Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	**Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1	Apply core concepts and techniques of artificial intelligence and data science to analyze complex datasets, extract meaningful insights, and develop intelligent systems for real-world applications, while demonstrating effective **teamwork** and **leadership** with **collaborative** problem-solving abilities.
PSO2	Demonstrate the ability to collect, process, analyze, and interpret large-scale data using modern tools and technologies to support data-driven decision-making for sustainable solutions.
PSO3	Exhibit the capability to design and develop solutions for varied application

	domains through self and lifelong learning abilities along with following professional **ethics,** responsibility and awareness of **societal** and environmental impacts when working with AI and data-driven technologies ensuring **privacy**, **fairness**, and transparency.

RUBRICS FOR LAB

Laboratory Evaluation Rubrics:

S. No.	Criteria	Sub Criteria and Marks Distribution			Outstanding (>90%)	Admirable (70-90%)	Average (40-69%)	Inadequate (<40%)
		Mid-Term	End-Team	Continues Evaluation				
A		Procedure Followed M.M. 100 = 6	Procedure Followed M.M. 100 = 6	Procedure Followed M.M. 100 = 2	● All possible system and Input/ Output variables are taken into account ● Performance measures are properly defined ● Experimental scenarios are very well defined	● Most of the system and Input/ Output variables are taken into account ● Most of the Performance measures are properly defined ● Experimental scenarios are defined correctly	● Some of the system and Input/ Output variables are taken into account ● Some of the Performance measures are properly defined ● Experimental scenarios are defined but not sufficient	● System and Input/ Output variables are not defined ● Performance measures are not properly defined · Experimental scenarios not defined
		Individual/Team Work M.M. 100 = 6	Individual/Team Work M.M. 100 = 6	Individual/Team Work M.M. 100 = 2	● Coordination among the group members in performing the experiment was excellent	● Coordination among the group members in performing the experiment was good	● Coordination among the group members in performing the experiment was average	● Coordination among the group members in performing the experiment was very poor
		Precision in data collection M.M. 100 = 6	Precision in data collection M.M. 100 = 6	Precision in data collection M.M. 100 = 4	● Data collected is correct in size and from the experiment performed	● Data collected is appropriate in size and but not from proper sources.	● Data collected is not so appropriate in size and but from proper sources.	● Data collected is neither appropriate in size and norfrom proper sources
B	LABRECORD/W	NA	NA	Timing of Evaluation of Experiment M.M. 100 = 6	● On the Same Date of Performance	● On the Next Turn from Performance	● Before Dead Line	● On the Dead Line

	RITTENWORK (PO1, PO8, PO10)	Data Analysis M.M. 100 = 6	Data Analysis M.M. 100 = 6	Data Analysis M.M. 100 = 4	• Data collected is exhaustively analyzed & appropriate features are selected	• Data collected is analyzed & but appropriate features are not selected	• Data collected is not analyzed properly. •Features selected are not appropriate	• Data collected is not analyzed & the features are not selected

		Results and Discussion M.M. 100 = 6	Results and Discussion M.M. 100 = 6	Results and Discussion M.M. 100 = 4	• All results are very well presented with all variables • Well prepared neat diagrams/plots/ tables for all performance measured • Discussed critically behavior of the system with reference to performance measures • Very well discussed pros n cons of outcome	• All results presented but not all variables mentioned • Prepared diagrams /plots/ tables for all performance measured but not so neat • Discussed behavior of the system with reference to performance measures but not critical • Discussed pros n cons of outcome in brief	• Partial results are included • Prepared diagrams /plots/ tables partially for the performance measures • Behavior of the system with reference to performance measures has been superficially presented • Discussed pros n cons of outcome but not so relevant	• Results are included but not as per experimental scenarios • No proper diagrams /plots/ tables are prepared • Behavior of the system with reference to performance measures has not been presented • Did not discuss pros n cons of outcome
		Way of presentation M.M. 100 = 5	Way of presentation M.M. 100 = 5	Way of presentation M.M. 100 = 4	• Presentation was very good	• Presentation was good	• Presentation was satisfactory	• Presentation was poor

C	**(I (I , I (I ( )**	Concept Explanation M.M. 100 = 5	Concept Explanation M.M. 100 = 5	Concept Explanation M.M. 100 = 4	• Conceptual explanation was excellent	• Conceptual explanation was good	• Conceptual explanation was somewhat good	• Conceptual explanation was Poor
D	**AT T E N D A N C E**	NA	NA	**A**ttendance M.M. 100 =10	• Present more than 90% of lab sessions	• Present more than 75% of lab sessions	• Present more than 60% of lab sessions	• Present in less than 60% lab sessions

LAB CONDUCTION PLAN

Total number of Experiments –15
Total numbers of turns required - 15

Experiment Number	Scheduled Week
Experiment -1	Week 1
Experiment -2	Week 2
Experiment -3	Week 3
Experiment -4	Week 4
Experiment -5	Week 5
Experiment -6	Week 6
Experiment -7	Week 7
I Mid Term	**Week 8**
Experiment -8	Week 9
Experiment-9	Week 10
Experiment-10	Week 11
Experiment-11	Week 12
Experiment-12	Week 13
Experiment-13	Week 14
II Mid Term	**Week 15**

DISTRIBUTION OF LAB HOURS

S. No.	Activity	Distribution of Lab Hours	
		Time (180 minute)	Time (120 minute)
1	Attendance	5	5
2	Explanation of Experiment & Logic	30	30
3	Performing the Experiment	60	30
4	File Checking	40	20
5	Viva/Quiz	30	20
6	Solving of Queries	15	15

LAB ROTAR PLAN

ROTOR-1

Ex. No.	NAME OF EXPERIMENTS
1	Zero Lab
2	Study of Different Type of LAN & Network Equipments.
3	Study and Verification of standard Network topologies i.e. Star, Bus, Ring etc.
4	LAN installations and Configurations.
5	Write a program to implement various types of error correcting techniques.
6	Write a program to implement various types of framing methods.
7	Write two programs in C: hello_client and hello_server a. The server listens for, and accepts, a single TCP connection; it reads all the data it can from that connection, and prints it to the screen; then it closes the connection b. The client connects to the server, sends the string "Hello, world!", then closes the connection
8	Repeat Exercises 6 & 7 for UDP.
9	Repeat Exercise 7 with multiplexed I/O operations.

ROTOR-2

Ex. No.	NAME OF EXPERIMENTS
10	Simulate Bellman-Ford Routing algorithm in NS2.
11	Use of Basic Unix Shell Commands: ls, mkdir, rmdir, cd, cat, banner, touch, file, wc, sort, cut, grep, dd, dfspace, du, ulimit.
12	Commands related to inode, I/O redirection and piping, process control commands, mails.
13	Shell Programming: Shell script based on control structure- If-then-fi, if-then else-if, nested if-else, to find: 1. Greatest among three numbers. 2. To find a year is leap year or not. 3. To input angles of a triangle and find out whether it is valid triangle or not. 4. To check whether a character is alphabet, digit or special character. 5. To calculate profit or loss.
14	Write a shell script to print different shapes- Diamond, triangle, square, rectangle, hollow square etc.
15	Shell Programming – Arrays.
16	Construct a network of hub and switch using Packet Tracer. **(Beyond Syllabus)**
17	Construct a network of Bus, star topolgy using Packet Tracer. **(Beyond Syllabus)**

GENERAL LAB INSTRUCTIONS

DO'S

1. Enter the lab on time and leave at proper time.
2. Wait for the previous class to leave before the next class enters.
3. Keep the bag outside in the respective racks.

Utilize lab hours in the corresponding.

4. Turn off the machine before leaving the lab unless a member of lab staff has specifically told you not to do so.
5. Leave the labs at least as nice as you found them.
6. If you notice a problem with a piece of equipment (e.g., a computer doesn't respond) or the room in general (e.g., cooling, heating, lighting) please report it to lab staff immediately. Do not attempt to fix the problem yourself.

DON'TS

1. Don't abuse the equipment.
2. Do not adjust the heat or air conditioners. If you feel the temperature is not properly set, inform lab staff; we will attempt to maintain a balance that is healthy for people and machines.
3. Do not attempt to reboot a computer. Report problems to lab staff.
4. Do not remove or modify any software or file without permission.
5. Do not remove printers and machines from the network without being explicitly told to do so by lab staff.
6. Don't monopolize equipment. If you're going to be away from your machine for more than 10 or 15 minutes, log out before leaving. This is both for the security of your account, and to ensure that others are able to use the lab resources while you are not.
7. Don't use internet, internet chat of any kind in your regular lab schedule.
8. Do not download or upload of MP3, JPG or MPEG files.
9. No games are allowed in the lab sessions.
10. No hardware including USB drives can be connected or disconnected in the labs without prior permission of the lab in-charge.
11. No food or drink is allowed in the lab or near any of the equipment. Aside from the fact that it leaves a mess and attracts pests, spilling anything on a keyboard or other piece of computer equipment could cause permanent, irreparable, and costly damage. (and in fact *has*) If you need to eat or drink, take a break and do so in the canteen.
12. Don't bring any external material in the lab, except your lab record, copy and books.

13. Don't bring the mobile phones in the lab. If necessary, then keep them in silence mode.
14. Please be considerate of those around you, especially in terms of noise level. While labs are a natural place for conversations of all types, kindly keep the volume turned down.
15. If you are having problems or questions, please go to either the faculty, lab in-charge or the lab supporting staff. They will help you. We need your full support and cooperation for smooth functioning of the lab.

LAB SPECIFIC SAFETY RULES

Before entering in the lab

1. All the students are supposed to prepare the theory regarding the next experiment/ Program.
2. Students are supposed to bring their lab records as per their lab schedule.
3. Previous experiment/program should be written in the lab record.
4. If applicable trace paper/graph paper must be pasted in lab record with proper labeling.
5. All the students must follow the instructions, failing which he/she may not be allowed in the lab.

While working in the lab

1. Adhere to experimental schedule as instructed by the lab in-charge/faculty.
2. Get the previously performed experiment/ program signed by the faculty/ lab in charge.
3. Get the output of current experiment/program checked by the faculty/ lab in charge in the Lab copy.
4. Each student should work on his/her assigned computer at each turn of the lab.
5. Take responsibility of valuable accessories.

Zero Lab

Introduction about Lab

Software Required
Turbo C, Linux, Packet Tracer

Socket

Sockets allow communication between two different processes on the same or different machines. To be more precise, it's a way to talk to other computers using standard Unix file descriptors. In Unix, every I/O action is done by writing or reading a file descriptor. A file descriptor is just an integer associated with an open file and it can be a network connection, a text file, a terminal, or something else.

To a programmer, a socket looks and behaves much like a low-level file descriptor. This is because commands such as read() and write() work with sockets in the same way they do with files and pipes.

Sockets were first introduced in 2.1BSD and subsequently refined into their current form with 4.2BSD. The sockets feature is now available with most current UNIX system releases

Where is Socket Used?

A Unix Socket is used in a client-server application framework. A server is a process that performs some functions on request from a client. Most of the application-level protocols like FTP, SMTP, and POP3 make use of sockets to establish connection between client and server and then for exchanging data.

Socket Types

There are four types of sockets available to the users. The first two are most commonly used and the last two are rarely used.

Processes are presumed to communicate only between sockets of the same type but there is no restriction that prevents communication between sockets of different types.

•**Stream Sockets** − Delivery in a networked environment is guaranteed. If you send through the stream socket three items "A, B, C", they will arrive in the same order − "A, B, C". These sockets use TCP (Transmission Control Protocol) for data transmission. If delivery is impossible, the sender receives an error indicator. Data records do not have any boundaries.

•**Sockets** − Delivery in a networked environment is not guaranteed. They're connectionless because you don't need to have an open connection as in Stream Sockets − you build a packet with the destination information

and send it out. They use UDP (User Datagram Protocol).

•**Raw Sockets** − These provide users access to the underlying communication protocols, which support socket abstractions. These sockets are normally datagram oriented, though their exact characteristics are dependent on the interface provided by the protocol. Raw sockets are not intended for the general user; they have been provided mainly for those interested in developing new communication protocols, or for gaining access to some of the more cryptic facilities of an existing protocol.

•**Sequenced Packet Sockets** − They are similar to a stream socket, with the exception that record boundaries are preserved. This interface is provided only as a part of the Network Systems (NS) socket abstraction, and is very important in most serious NS applications. Sequenced-packet sockets allow the user to manipulate the Sequence Packet Protocol (SPP) or Internet Datagram Protocol (IDP) headers on a packet or a group of packets, either by writing a prototype header along with whatever data is to be sent, or by specifying a default header to be used with all outgoing data, and allows the user to receive the headers on incoming packets.

Unix Socket - Client Server Model

Most of the Net Applications use the Client-Server architecture, which refers to two processes or two applications that communicate with each other to exchange some information. One of the two processes act as a client process, and another process acts as a server.

Client Process

This is the process, which typically makes a request for information. After getting the response, this process may terminate or may do some other processing.

Example, Internet Browser works as a client application, which sends a request to the Web Server to get one HTML webpage.

Server Process

This is the process which takes a request from the clients. After getting a request from the client, this process will perform the required processing, gather the requested information, and send it to the client. Once done, it becomes ready to serve another client. Server processes are always alert and ready to serve incoming requests.

Example − Web Server keeps waiting for requests from Internet Browsers and as soon as it gets any request from a browser, it picks up a requested HTML page and sends it back to that Browser.

Note that the client needs to know the address of the server, but the server does not need to know the address or

even the existence of the client prior to the connection being established. Once a connection is established, both sides can send and receive information.

Architectures

There are two types of client-server architectures −

•2-tier architecture − In this architecture, the client directly interacts with the server. This type of architecture may have some security holes and performance problems. Internet Explorer and Web Server work on two-tier architecture. Here security problems are resolved using Secure Socket Layer (SSL).

•3-tier architectures − In this architecture, one more software sits in between the client and the server. This middle software is called 'middleware'. Middleware are used to perform all the security checks and load balancing in case of heavy load. A middleware takes all requests from the client and after performing the required authentication, it passes that request to the server. Then the server does the required processing and sends the response back to the middleware and finally the middleware passes this response back to the client. If you want to implement a 3-tier architecture, then you can keep any middleware like Web Logic or WebSphere software in between your Web Server and Web Browser.

Types of Server

There are two types of servers you can have −

•Iterative Server − This is the simplest form of server where a server process serves one client and after completing the first request, it takes request from another client. Meanwhile, another client keeps waiting.

•Concurrent Servers − This type of server runs multiple concurrent processes to serve many requests at a time because one process may take longer and another client cannot wait for so long. The simplest way to write a concurrent server under Unix is to fork a child process to handle each client separately.

EXPERIMENT-1

OBJECTIVE

To study of different type of LAN &Network Equipment's.

Local Area Network (LAN)

We're confident that you've heard of these types of networks before – LANs are the most frequently discussed networks, one of the most common, one of the most original and one of the simplest types of networks. LANs connect groups of computers and low-voltage devices together across short distances (within a building or

between a group of two or three buildings in close proximity to each other) to share information and resources. Enterprises typically manage and maintain LANs.
Using routers, LANs can connect to wide area networks (WANs, explained below) to rapidly and safely transfer data.

Metropolitan Area Network (MAN)

These types of networks are larger than LANs but smaller than WANs – and incorporate elements from both types of networks. MANs span an entire geographic area (typically a town or city, but sometimes a campus). Ownership and maintenance is handled by either a single person or company (a local council, a large company, etc.).

Wide Area Network (WAN)

Slightly more complex than a LAN, a WAN connects computers together across longer physical distances. This allows computers and low-voltage devices to be remotely connected to each other over one large network to communicate even when they're miles apart.
The Internet is the most basic example of a WAN, connecting all computers together around the world. Because of a WAN's vast reach, it is typically owned and maintained by multiple administrators or the public.
Different Networking Equipment —

Network Hub:

Network Hub is a networking device which is used to connect multiple network hosts. A network hub is also used to do data transfer. The data is transferred in terms of packets on a computer network. So when a host sends a data packet to a network hub, the hub copies the data packet to all of its ports connected to. Like this, all the ports know about the data and the port, for whom the packet is intended, claims the packet.

However, because of its working mechanism, a hub is not so secure and safe. Moreover, copying the data packets on all the interfaces or ports makes it slower and more congested which led to the use of network switch.

Network Switch:

Like a hub, a switch also works at the layer of LAN (Local Area Network) but you can say that a switch is more intelligent than a hub. While hub just does the work of data forwarding, a switch does 'filter and forwarding' which is a more intelligent way of dealing with the data packets.

So, when a packet is received at one of the interfaces of the switch, it filters the packet and sends only to the interface of the intended receiver. For this purpose, a switch also maintains a CAM (Content Addressable Memory) table and has its own system configuration and memory. CAM table is also called as forwarding table or forwarding information base (FIB).

Modem:

A Modem is somewhat a more interesting network device in our daily life. So if you have noticed around, you get an internet connection through a wire (there are different types of wires) to your house. This wire is used to carry our internet data outside to the internet world.
However, our computer generates binary data or digital data in forms of 1s and 0s and on the other hand, a wire carries an analog signal and that's where a modem comes in.
A modem stands for (Modulator+Demodulator). That means it modulates and demodulates the signal between the digital data of a computer and the analog signal of a telephone line.

Network Router:

A router is a network device which is responsible for routing traffic from one to another network. These two networks could be a private company network to a public network. You can think of a router as a traffic police who directs different network traffic to different directions.

Bridge:

If a router connects two different types of networks, then a bridge connects two subnetworks as a part of the same network. You can think of two different labs or two different floors connected by a bridge.

Repeater:

A repeater is an electronic device that amplifies the signal it receives. In other terms, you can think of repeater as a device which receives a signal and retransmits it at a higher level or higher power so that the signal can cover longer distances.
For example, inside a college campus, the hostels might be far away from the main college where the ISP line comes in. If the college authority wants to pull a wire in between the hostels and main campus, they will have to use repeaters if the distance is much because different types of cables have limitations in terms of the distances they can carry the data for.

When these network devices take a particular configurational shape on a network, their configuration gets a particular name and the whole formation is called Network topology. In certain circumstances when we add some more network devices to a network topology, its called Daisy chaining.

Figure 1: Different network equipment

Viva Questions:

1) Differentiate between Hub, Switch and Router.
2) Differentiate between Router and Gateway.
3) What is Broadcast domain and Collision domain?
4) Compare Hub and Switch with respect to Broadcast domain and Collision domain?
5) How does Switch perform forwarding function.

EXPERIMENT-2

OBJECTIVE

Study and Verification of standard Network topologies i.e. Star, Bus, Ring etc

Network Topologies:

LAN physical topology defines the geographical arrangement of networking devices. Topologies are driven fundamentally by two network connection types. A point-to-point connection is a direct link between two

devices. For example, when you attach your computer to a printer, you have created a point-to-point link. In networking terms, most of the today's point-to-point connections are associated with modems and PSTN (Public Switched Telephone Network) communications because only two devices share point-to-point connections, it defeats the purpose of a shared network.

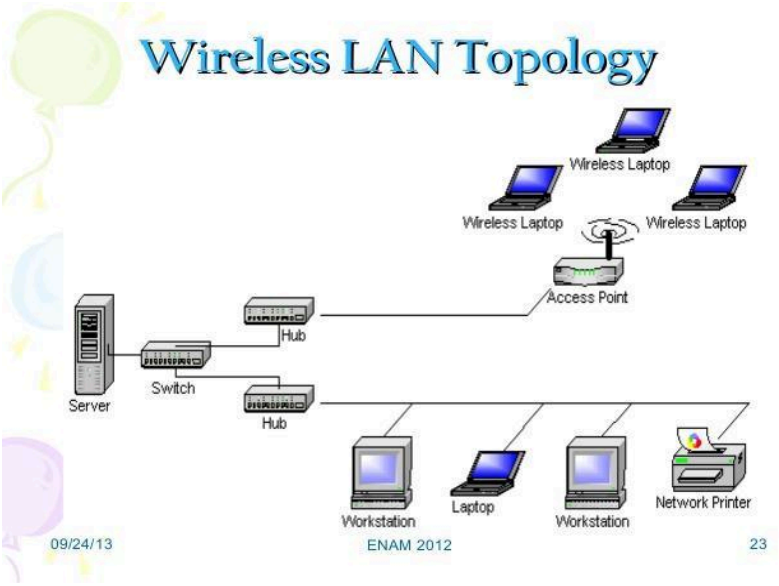


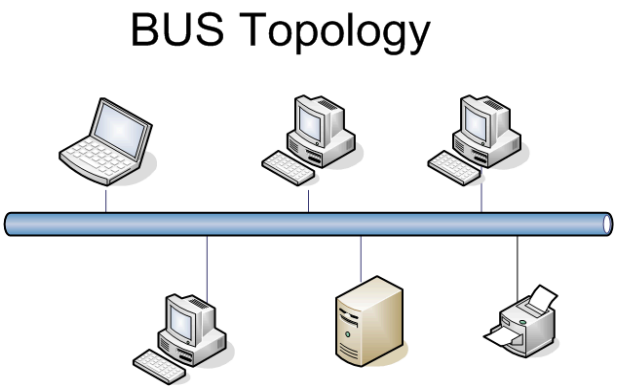
Figure 2: Pictorial Description of LAN Topology

The major topologies of LAN are:

Bus Topology:

The physical bus topology is the simplest and most widely used of the network designs. It consists of one continuous length of cabling (trunk) and a terminating resistor (terminator) at each end. The data communications message travels along the bus in both directions until it is picked up by a workstation or server NIC.

If the message is missed or not recognized, it reaches the end of the cabling and dissipates at the terminator. All nodes in the bus topology have equal access to the trunk no discriminating here. This is accomplished using short drop cables or direct T-connectors.
This design is easy to install because the backbone trunk traverses the LAN as one cable segment. This minimizes the amount of transmission media required. Also, the number of devices and length of the trunk can be easily expanded. Advantages of Bus Topology: It uses established standards and it is relatively easy to install.



Advantages of Bus Topology:

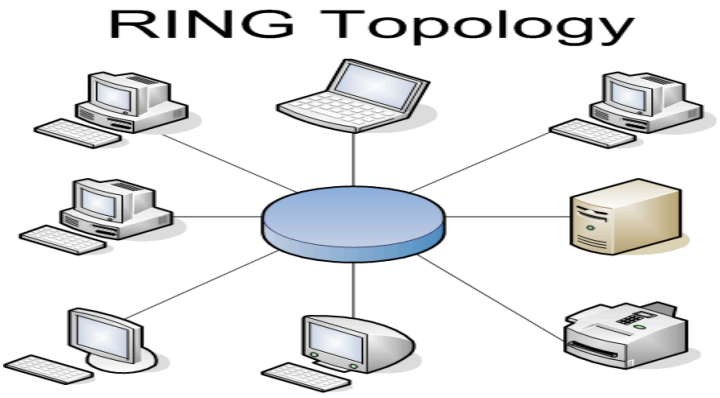
- It uses established standards and it is relatively easy to install.
- It requires fewer media than other topologies.

Disadvantages of Bus Topology:

- The bus networks are difficult to reconfigure, especially when the acceptable number of connections or maximum distances have been reached.
- They are also difficult to troubleshoot because everything happens on a single media segment. This can have dangerous consequences because any break in the cabling brings the network to its knees.

Ring Topology

As its name implies, the physical ring topology is a circular loop of point-to-point links. Each device connects directly or indirectly to the ring through an interface device or drop cable. Messages travel around the ring from node to node in very organized manner. Each workstation checks the messages for a matching destination address.



If the address doesn't match, the node simply regenerates the message and sends it on its way. If the address matches, the node accepts the message and sends a reply to the originating sender. Initially, ring topologies are

moderately simple to install; however, they require more media than bus systems because the loop must be closed. Once your ring has been installed, it's a bit more difficult to reconfigure. Ring segments must be divided or replaced every time they are changed. Moreover, any break in the loop can affect all devices on the network.

Advantages of Ring Topology:

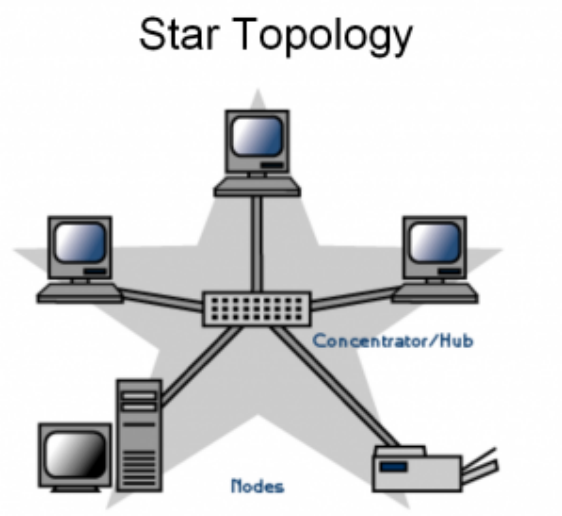
- They are very easy to troubleshoot because each device incorporates a repeater.
- A special internal feature called becoming, allows the troubled workstation to identify them quickly.

Disadvantages of Ring Topology:

- It is considerably difficult to install and reconfigure ring topology.
- Media failure on unidirectional or single loop causes complete network failure.

Star Topology:

The Physical star topology uses a central controlling hub with dedicated legs pointing in all directions like points of a star. Each network devices has a dedicated point-to-point link to the central hub. This strategy prevents troublesome collisions and keeps the line of communication open and free of traffic.



Star topologies are somewhat difficult to install because each device gets its own dedicated segment. Obviously, they require a great deal of cabling. This design provides an excellent platform for reconfiguration and troubleshooting. Changes to the network are as simple as plugging another segment into the hub. In addition, a break in the LAN is easy to isolate and doesn't affect the rest of the network.

Advantages of Star Topology:

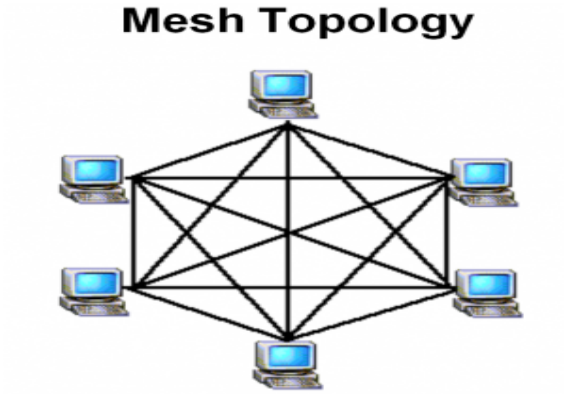
- Relatively easy to configure.
- Easy to troubleshoot.
- Media faults are automatically isolated to the failed segment.

Disadvantages of Star Topology:

- Requires more cable than most topologies.
- Moderately difficult to install.

Mesh Topology

The mesh topology is the only true point-to-point design. It uses a dedicated link between every device on the network. This design is not very practical because of its excessive waste of transmission media. This topology is difficult to install and reconfigure.



Moreover, as the number of devices increases geometrically, the speed of communication also becomes slow. ATM (Asynchronous Transfer Mode) and switched Hubs are the example of high-speed Mesh implementation.

Advantages of Mesh Topology:

- Easy to troubleshoot because each link is independent of all others.
- You can easily identify faults and isolate the affected links.
- Because of the high number of redundant paths, multiple links can fail before the failure affects any network device.

Disadvantages of Mesh Topology:

It is difficult to install and reconfigure especially as the number of devices increases.

Viva Questions

1) How many ports and cable links are required for a star topology?
2) For n devices in a network, what is the number of cable links required for a mesh, ring, bus topology.
3) What happens if a connection fails for devices arranged in a Mesh topology?
4) What is the consequence if connection fails for devices arranged in a Ring topology?
5) What occurs when a connection fails in a Star topology?

EXPERIMENT- 3

OBJECTIVE

LAN installations and Configurations.

Installation:

Count the number of computers you need to hardwire

When setting up a LAN, you'll need to know how many computers will be connecting to the network via Ethernet. This will determine the number of ports you'll need.
If you have four or fewer computers that you need to hardwire, you'll just need a router. If you have more than four, you'll likely need to get a switch to extend the number of ports available on your router.

Decide if you want to create a wireless network

If you want to allow devices to connect wirelessly, you'll need a router that can broadcast a wireless network. Most routers you'll find at the store or online have wireless capabilities.
Network switches do not allow wireless devices to connect, and can only be used for hardwired LANs or to extend the number of ports available to the router.

Determine if you want all network devices to have internet access

If you want all of the connected devices to have access to the internet, you'll need a router to handle the connections. If you don't need the devices to have a network connection, you can just use a network switch.

Measure the distances for all hardwired devices

This isn't much of an issue in most homes, but network cables cannot run longer than 100m (328 ft). If you have to run cable farther than this, you'll need switches in between.

Gather your network hardware

To create a LAN, you'll need a router or switch, which will act as the hub of your network. These devices route information to the correct computers.

A router will automatically handle assigning IP addresses to each device on the network, and is necessary if you intend to share your internet connection with all the connected devices. It is highly recommended that you build your network with a router, even if you're not sharing an internet connection.

A network switch is like a simpler version of a router. It will allow connected devices to talk to each other, but will not automatically assign IP addresses and will not share an internet connection. Switches are best used to expand the number of LAN ports available on the network, as they can be connected to the router.

Set up your router

You don't need to do much to set up a router for a basic LAN. Just plug it into a power source, preferably close to your modem if you plan on sharing the internet connection through it.

Connect your modem to your router (if necessary)

If you're sharing the internet connection from your modem, connect the modem to the WAN/INTERNET port on the router. This is usually a different color from the other ports.

Connect your switch to your router (if necessary).

If you're using a switch to expand the number of ports available on the router, plug an Ethernet cable into any LAN port on the router and any LAN port on the switch. This will expand the network to the rest of the LAN ports on the switch.

Connect your computers to open LAN ports

Use Ethernet cables to connect each computer to an open LAN port on your router or switch. It doesn't matter what order the ports are connected in. Ethernet cables cannot reliably transfer data at lengths larger than 100m (328 ft).

Setup one PC as a DHCP server if you're just using a switch

If you're only using a switch as your network hub, setting up one computer as a DHCP (Dynamic Host Configuration Protocol) server will allow all of the connected computers to easily obtain IP addresses.
You can quickly create a DHCP server on one of your computers by installing a third-party utility.
The rest of the computers on the network will obtain IP addresses automatically once the server is running, as long as they are set to do so.

Verify the network connection on each computer

After each computer obtains an IP address, they'll be able to talk to each other on the network. If you're using a router to share your internet connection, each computer will be able to access the internet.

Verify the network connection on each computer

After each computer obtains an IP address, they'll be able to talk to each other on the network. If you're using a router to share your internet connection, each computer will be able to access the internet.

Set up file and printer sharing

Once your network is up, you won't see anything on other computers unless that computer has shared files. You can designate files, folders, drives, printers, and other devices as shared so that anyone on the network, or just specific users, can access them.

Set up your router

When you're setting up a wireless router, you'll need to keep a few things in mind: For easy troubleshooting, the router should usually be placed close to your modem. It should be located centrally to allow for maximum wireless coverage. You'll need to connect a computer to the router via Ethernet during the setup process.

Plug a computer into one of the router's LAN ports

You'll be using your computer's web browser to configure the router's wireless network.
Type in the router's IP address

You can typically find this printed on the bottom of the router, or in your router's documentation. If you can't find it, there are a couple things you can try:
Windows - Right-click the Network button in the System Tray → click Open Network and Sharing Center → click the Ethernet link → click Details → find the Default Gateway entry for your router's IP address.
Mac - Click the Apple menu and select System Preferences → click Network → click your Ethernet connection → find the Router entry for your router's IP address.

Log in with the administrator account

You'll be prompted for the login information for your router. The default login information varies depending on your router model, but the username is often "admin" and the password is often "admin," "password," or blank.
You can look up your router model at https://portforward.com/router-password/ to find the default login information.
Open the **Wireless** section of the router settings. The exact location and wording of this section varies from router to router.
Change the name of your network in the **SSID field**. This may also be called "Network name." This is the

name that appears in the list of available wireless networks.
Select **WPA2-Personal** as the Authentication or Security option. This is the most secure option currently available on most routers. Avoid WPA and WEP except when explicitly required by older, incompatible devices.

Ensure the wireless network is enabled

Depending on the router, you may have to check a box or click a button at the top of the Wireless menu to enable the wireless network.

Configuration:

Determine the number of computers you want to connect

The number of computers you're connecting will determine the type of network hardware you'll need. If you are connecting four or less computers, you'll just need a single router, or one switch if you don't need internet. If you're connecting more than four computers, you'll want a router and a switch, or just a switch if you don't need internet.

Determine your network layout

If you're installing a permanent LAN solution, you'll want to keep cable length in mind. CAT5 Ethernet cables should not run longer than 250 feet. If you need to cover larger distances, you'll need switches at regular intervals, or you'll need to use CAT6 cables.
You'll need one Ethernet cable for each computer you want to connect to the LAN, as well as an Ethernet cable to connect the router to the switch (if applicable).

Obtain the network hardware

To create a LAN, you'll need a router and/or a network. These pieces of hardware are the "hub" of your LAN, and all of your computers will be connected to them.
The easiest way to create a LAN where every computer has access to the internet is to use a router, and then add a network switch if the router doesn't have enough ports. A router will automatically assign an IP address to every computer that is connected to it.
Switches are similar to routers but do not automatically assign IP addresses. Switches typically have many more Ethernet ports than a router has.

Connect your modem to the WAN port on the router

This port may be labeled "INTERNET" instead. This will provide internet access to every computer that is connected to your LAN.
You can skip this if you're setting up a LAN without internet access. You don't need a router at all to create a LAN, but it makes things easier. If you just use a network switch, you'll need to manually assign IP addresses to each computer after connecting them.

Connect the switch to a LAN port on the router

If you're using a network switch to connect more computers, connect it to one of the LAN ports on the router. You can use any open port on the switch to make the connection. When connected, the router will provide IP addresses for every computer that is connected to either device.

Find the Ethernet port on your PC

You can usually find this on the back of your desktop tower, or along the side or back of a laptop. Slim laptops may not have an Ethernet port, in which case you'll need to either use a USB Ethernet adapter or connect wirelessly if your router allows it.

Plug one end of an Ethernet cable into your computer

Make sure you're using an Ethernet cable (RJ45), not a telephone cable (RJ11).
Plug the other end of the cable into an open LAN port

This can be any open LAN port on either the router or the switch, depending on your LAN setup.

Test out your network (router only)

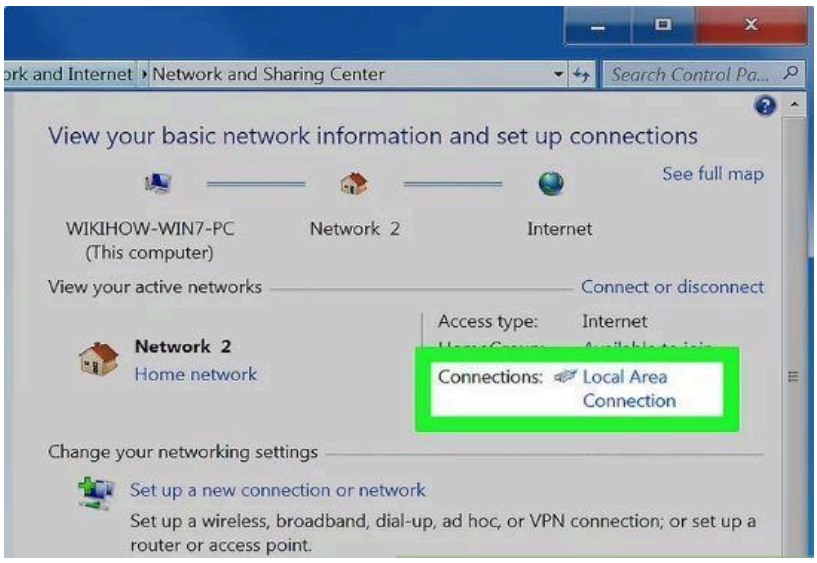
If you're using a router, your work is complete. Once all of the computers are connected to a LAN port, they will be assigned IPs automatically and will appear on the network. If you set up your LAN for gaming, you should be able to start your LAN game and have each computer connect. If you're using a switch and no router, you'll still need to assign IP addresses to each computer.

Right-click on your network connection

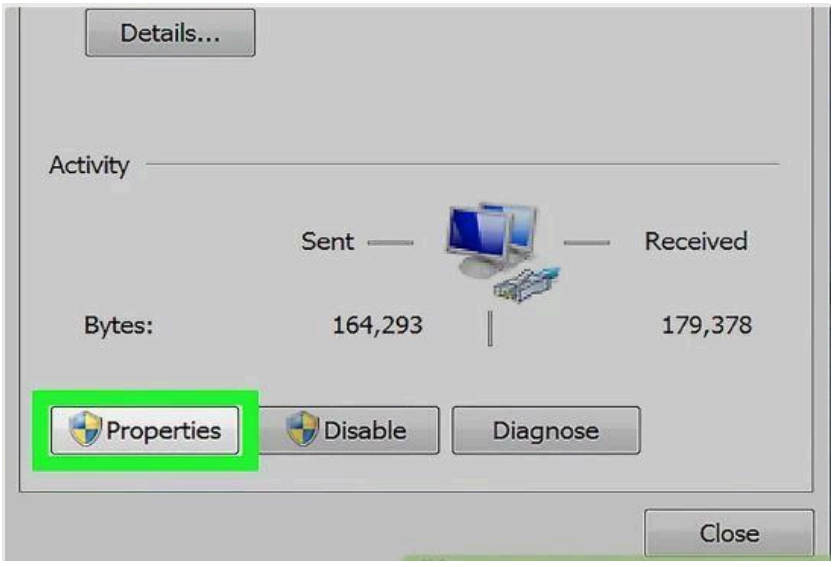
You'll see this in your System Tray. If you are connecting your computers through a switch with no router, you'll need to assign each computer on the network its own individual IP address. This process is handled automatically if you're using a router.

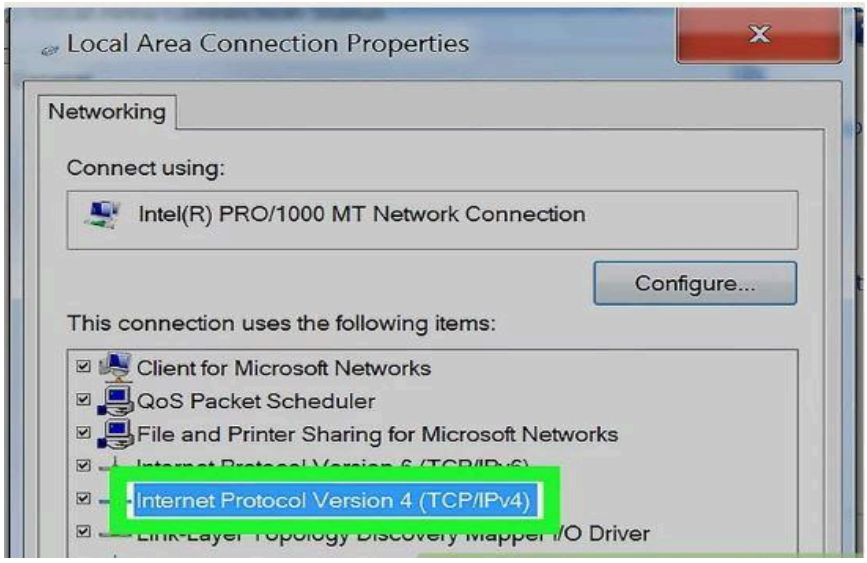
Think of an IP address as a mailing address. Each computer on the network needs a unique IP address so that information sent across the network reaches the correct destination.

Click Open Network and Sharing Center



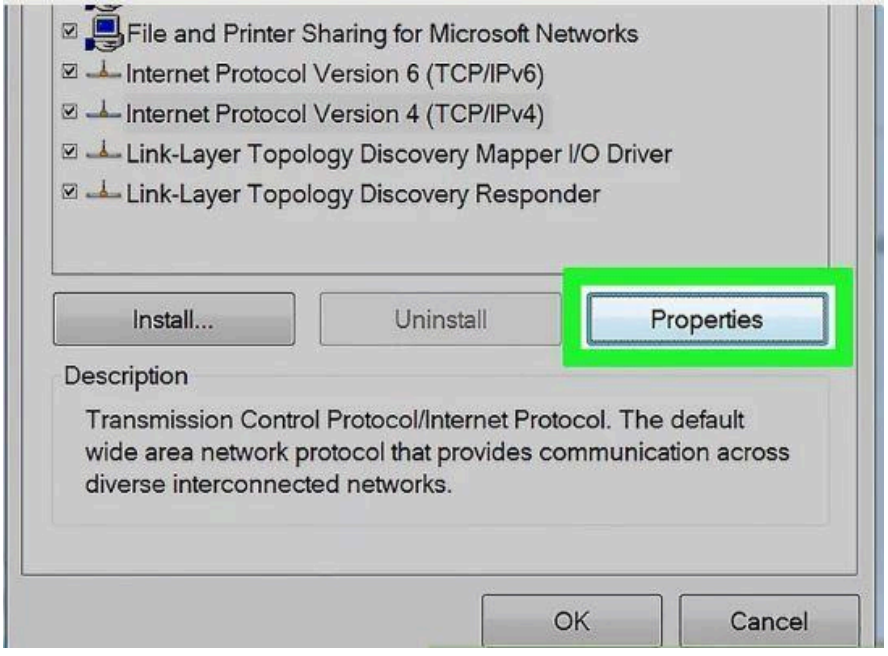
Click the Ethernet link at the top of the window. You'll see this next to "Connections."



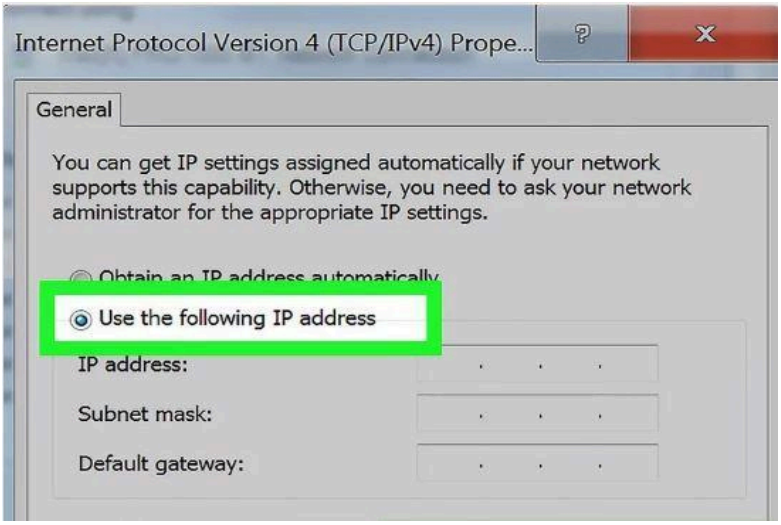


Click Internet Protocol Version 4 (TCP/IPv4).

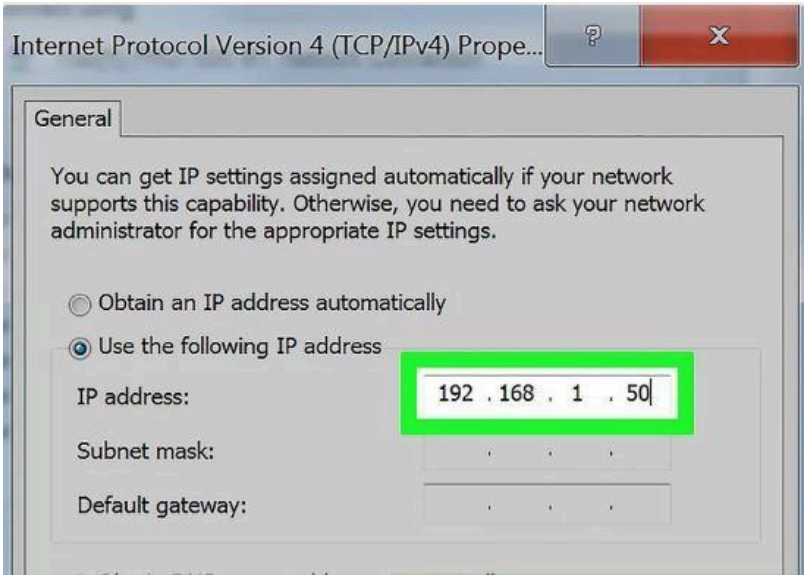
Make sure you don't uncheck it, just highlight it.



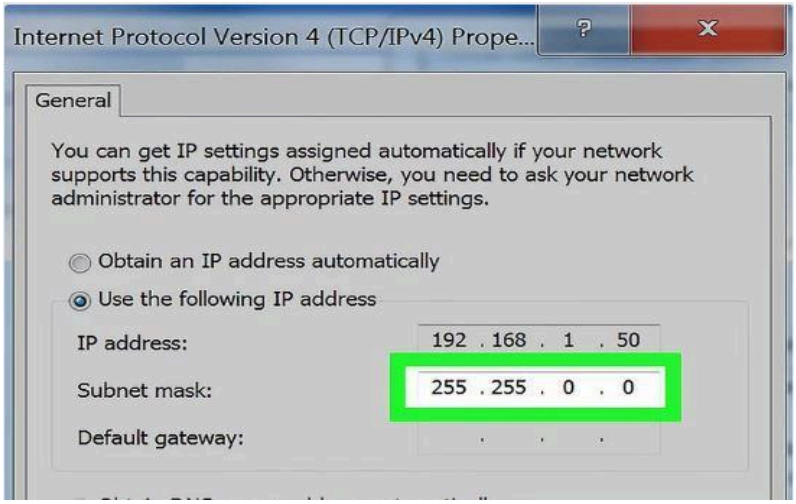
Click Properties.



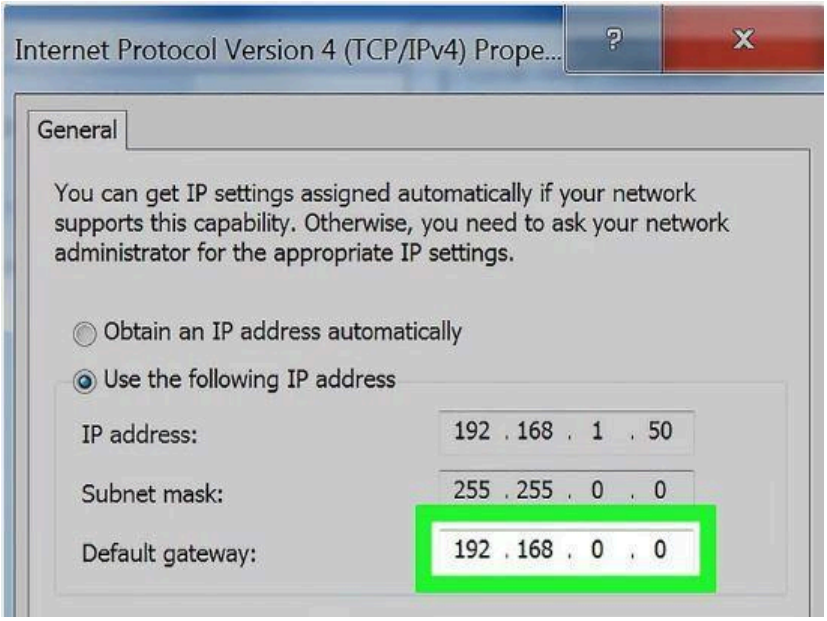
Click the Use the following IP address radio button.



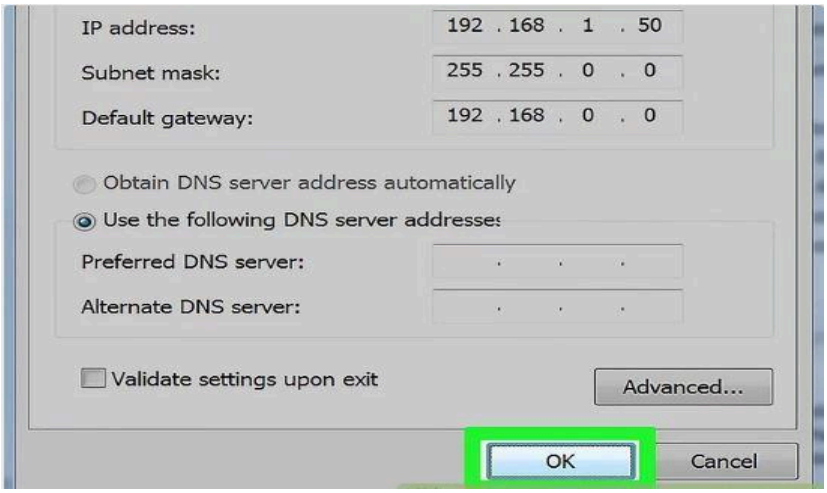
Type 192.168.1.50 into the IP address field.



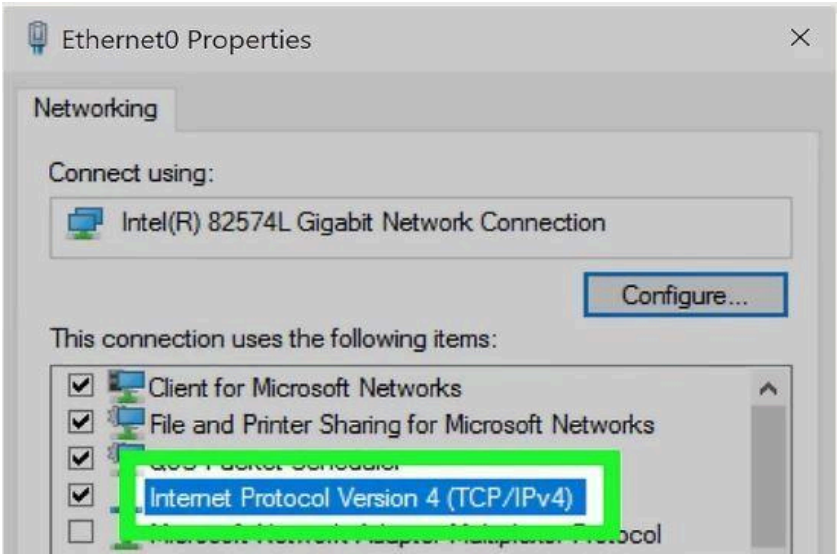
Type `255.255.0.0` into the Subnet mask field.



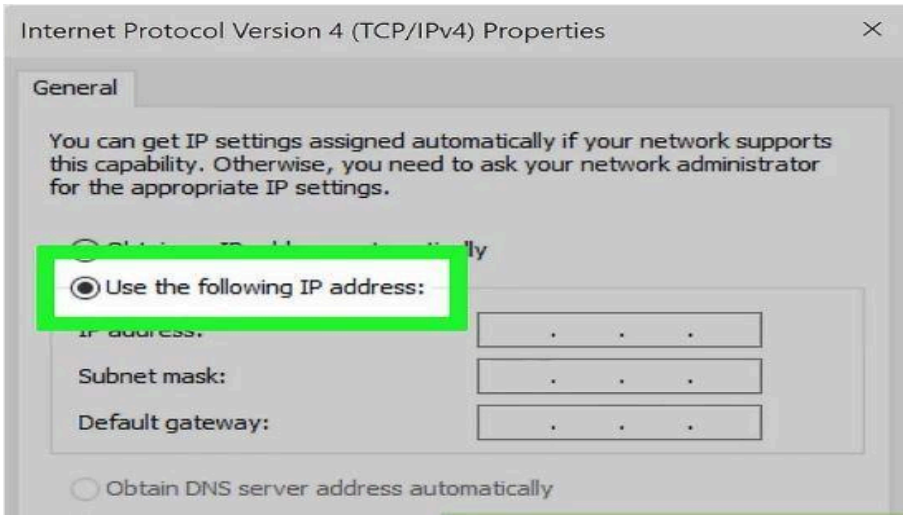
Type `192.168.0.0` into the Default gateway field.



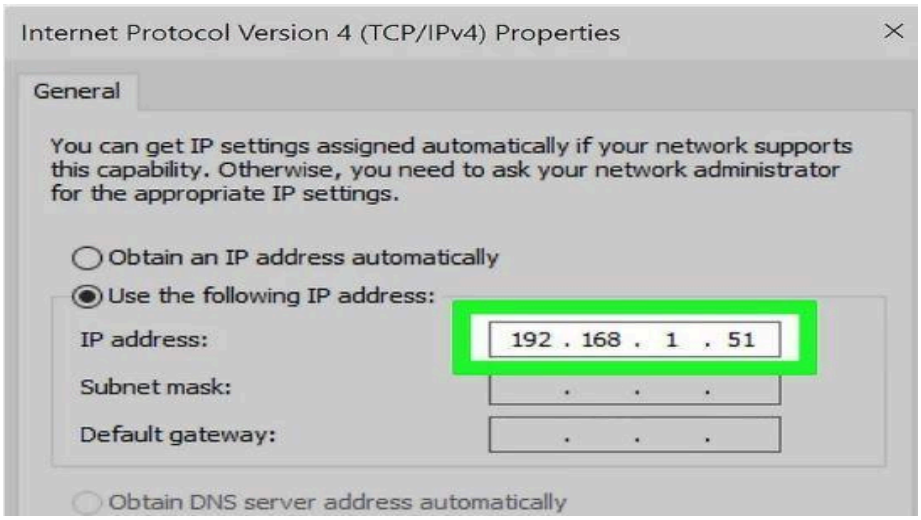
Click OK. This will save the settings for that computer. This computer is now configured on your network with a unique IP address.



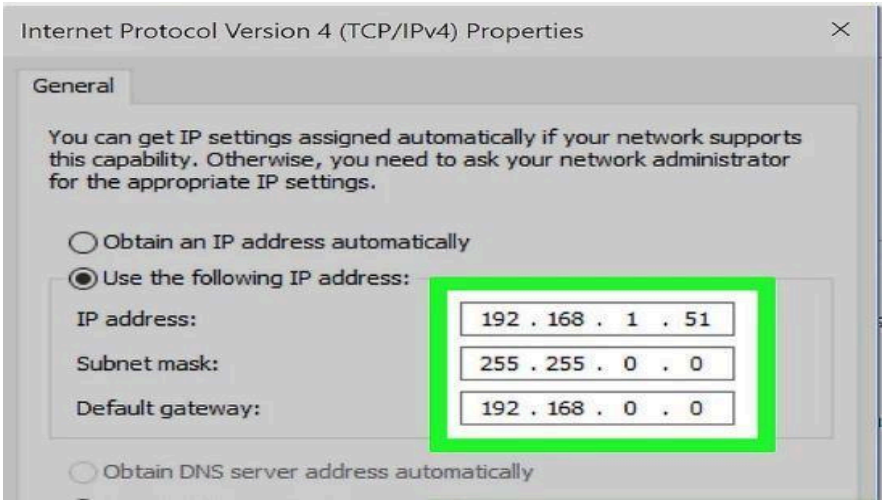
Open the Internet Protocol Version 4 properties on the next computer. Follow the steps above on the second computer to open the Internet Protocol Version 4 (TCP/IPv4) Properties window.



Click the Use the following IP address radio button.



Type `192.168.1.51` into the IP address field. Notice that the final group of numbers has incremented by 1.



Enter the same values for Subnet mask and Default gateway. These values should be the same as they were on the first computer (255.255.0.0 and 192.168.0.0 respectively).

Viva Questions:

1) What is the difference between a public and private IP address?
2) A combination of hardware and software which links two parts of the same LAN or two compatible LANs by directing messages to the correct node is called.
3) What are the ranges of Class A, Class B, Class C, Class D and Class E.
4) Mention the starting address of Class A, Class B, Class C, Class D and Class E.
5) Mention the ending address of Class A, Class B, Class C, Class D and Class E.
6) Mention the addresses per network of Class A, Class B, Class C, Class D and Class E.
7) Mention the number of networks of Class A, Class B, Class C, Class D and Class E.
8) How many net id bits and host id bits in Class A, Class B, Class C, Class D and Class E.

EXPERIMENT- 4

OBJECTIVE

Write a program to implement various types of error correcting techniques.

PROGRAM:

```
#include<iostream>
using namespace std;
int main()
{
int a[10],b[10],c1,c2,c3;
cout<<"\nEnter the 4 bits"; cin>>a[3];
cin>>a[5]; cin>>a[6]; cin>>a[7];
a[1]=a[3]^a[5]^a[7];
a[2]=a[3]^a[6]^a[7];
a[4]=a[5]^a[6]^a[7];
for(int i=1;i<8;i++)
cout<<"\t"<<a[i]; cout<<"\nEnter the 7 bits"; for(int i=1;i<8;i++)
{
cin>>b[i];
}
c1=b[1]^b[3]^b[5]^b[7];
c2=b[2]^b[3]^b[6]^b[7];
c3=b[4]^b[5]^b[6]^b[7];
int p=c1*1+c2*2+c3*4; if(p==0)
{

cout<<"\nThere is No error in the data";
}
else
{
cout<<"\nThere is error in the position "<<p;
if(b[p]==0)
b[p]=1;
else
b[p]=0;
for(int i=1;i<8;i++)
cout<<"\n"<<b[i];
}
return 0;}
```

OUTPUT

```
Enter 4 bits of data one by one
1
0
1
0
Encoded data is
1010010
Enter received data bits one by one
1
0
```

1
0
0
1
0
No error while transmission of data

Viva Questions:

1) Differentiate between single bit error, multiple bit error and burst bit error.
2) How CRC is different from checksum.
3) What is Hamming code correction method?
4) How can you perform access control in Data Link Layer?
5) How can you perform flow control in Data Link Layer?

EXPERIMENT-5

OBJECTIVE

Write a program to implement various types of framing methods.

PROGRAM

```
#include<stdio.h>
int main()
{
```

```
 int i=0,count=0;
 char databits[80];
 printf("Enter Data Bits: ");
 scanf("%s",databits);
 printf("Data Bits Before Bit Stuffing:%s",databits);
 printf("\nData Bits After Bit stuffing :");
 for(i=0; i<strlen(databits); i++)
 {
   if(databits[i]=='1')
      count++;
   else
      count=0;
printf("%c",databits[i]);
   if(count==5)
   {
      printf("0");
      count=0;
   }
 }
 return 0;
}
```

Output of the Program:
Enter Data Bits: 101111111000
Data Bits Before Bit Stuffing: 101111111000
Data Bits After Bit Stuffing : 1011111011000

Viva Questions:

1) What is the purpose of framing in the networking?
2) Mention the elements of frame.
3) Differentiate between bit stuffing and byte stuffing.
4) What is FRAD in frame relay network?
5) What are the methods of framing?

EXPERIMENT-6

OBJECTIVE

Write two programs in C: hello_client and hello_server

a. The server listens for, and accepts, a single TCP connection; it reads all the data it can from that connection, and prints it to the screen; then it closes the connection

b. The client connects to the server, sends the string "Hello, world!", then closes the connection

PROGRAM

a) **/* CLIENT PROGRAM FOR TCP CONNECTION */**

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

void error(const char *msg)

{
perror(msg);
exit(0);

}

int main(intargc, char *argv[])

{
        intsockfd, portno, n;
        structsockaddr_inserv_addr;
        structhostent *server;
        char buffer[256];

        if (argc< 3)
        {
fprintf(stderr,"usage %s hostname port\n", argv[0]);
exit(0);

        }
        portno = atoi(argv[2]);
sockfd = socket(AF_INET, SOCK_STREAM, 0);

        if (sockfd< 0)
        error("ERROR opening socket");

        server = gethostbyname(argv[1]);

        if (server == NULL)
        {

                fprintf(stderr,"ERROR, no such host\n");
        exit(0);
```

```
        }
bzero((char *) &serv_addr, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
bcopy((char *)server->h_addr,char *)&serv_addr.sin_addr.s_addr,
server->h_length);
serv_addr.sin_port = htons(portno);
if(connect(sockfd,(structsockaddr *) &serv_addr,sizeof(serv_addr)) < 0)

        error("ERROR connecting");

printf("Please enter the message: ");
bzero(buffer,256);
fgets(buffer,255,stdin);
n = write(sockfd,buffer,strlen(buffer));

if (n < 0)

        error("ERROR writing to socket");

bzero(buffer,256);
n = read(sockfd,buffer,255);
if (n < 0)
        error("ERROR reading from socket");

printf("%s\n",buffer);
close(sockfd);
return 0;

}
```

TO RUN-

```
$ gcc –o client helloclient.c
$ ./client localhostportno
```

OUTPUT

```
Please enter the message: I m client
I got your message
```

/* **SERVER PROGRAM FOR TCP CONNECTION***/

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
```

```
#include <netinet/in.h>

void error(const char *msg)
{
perror(msg);
exit(1);
}

int main(intargc, char *argv[])
{
intsockfd, newsockfd, portno;
socklen_tclilen;
char buffer[256];
structsockaddr_inserv_addr, cli_addr;
int n;
if (argc< 2) {
fprintf(stderr,"ERROR, no port provided\n");
exit(1);
    }
sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd< 0)
error("ERROR opening socket");
bzero((char *) &serv_addr, sizeof(serv_addr));
portno = atoi(argv[1]);
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = INADDR_ANY;
serv_addr.sin_port = htons(portno);
if (bind(sockfd, (structsockaddr *) &serv_addr,
sizeof(serv_addr)) < 0)
error("ERROR on binding");
listen(sockfd,5);
clilen = sizeof(cli_addr);
newsockfd = accept(sockfd,
            (structsockaddr *) &cli_addr,
&clilen);
if (newsockfd< 0)
error("ERROR on accept");
bzero(buffer,256);
    n = read(newsockfd,buffer,255);
if (n < 0) error("ERROR reading from socket");
printf("Here is the message: %s\n",buffer);
    n = write(newsockfd,"I got your message",18);
if (n < 0) error("ERROR writing to socket");
```

```
close(newsockfd);
close(sockfd);
return 0;
}
```

To RUN-

```
$ gcc –o server helloserver.c
$ ./server portno
```

OUTPUT

Here is the message: I'm client

Viva Questions:

1) Draw the diagram of three way handshake for TCP connection establishment.
2) How can you create TCP socket.
3) How can you bind the socket to the server address?
4) Mention the use of listen() method in TCP connection.
5) Mention the use of accept() method in TCP connection.

EXPERIMENT- 7

OBJECTIVE

Write an Echo_Client and Echo_server using TCP to estimate the round trip time from client to the server. The server should be such that it can accept multiple connections at any given time.

PROGRAM

```
//Echo server
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <stdio.h>
#include<string.h>
int main()
{

charstr[100];
intlisten_fd, comm_fd;

structsockaddr_inservaddr;

listen_fd = socket(AF_INET, SOCK_STREAM, 0);

bzero(&servaddr, sizeof(servaddr));

servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htons(INADDR_ANY);
servaddr.sin_port = htons(22000);

bind(listen_fd, (structsockaddr *) &servaddr, sizeof(servaddr));

listen(listen_fd, 10);
comm_fd = accept(listen_fd, (structsockaddr*) NULL, NULL);
```

```
while(1)
   {

bzero(str, 100);

read(comm_fd,str,100);

printf("Echoing back - %s",str);

write(comm_fd, str, strlen(str)+1);

   }
}
```

To Run:
$ gcc –o server echoserver.c
$ ./server

Output:

Output:
Echoing back – Hello
Echoing back – How r u
Echoing back – I m fine
Echoing back – Bye

```
//Client process
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <stdio.h>
#include<string.h>
int main(intargc,char **argv)
{
intsockfd,n;
charsendline[100];
charrecvline[100];
structsockaddr_inservaddr;

sockfd=socket(AF_INET,SOCK_STREAM,0);
bzero(&servaddr,sizeofservaddr);

servaddr.sin_family=AF_INET;
servaddr.sin_port=htons(22000);

inet_pton(AF_INET,"127.0.0.1",&(servaddr.sin_addr));

connect(sockfd,(structsockaddr *)&servaddr,sizeof(servaddr));
```

```
while(1)
   {
bzero(sendline, 100);
bzero(recvline, 100);
fgets(sendline,100,stdin); /*stdin = 0 , for standard input */

write(sockfd,sendline,strlen(sendline)+1);
read(sockfd,recvline,100);
printf("%s",recvline);
   }

}
```

To Run:

```
$ gcc –o client client1.c
$ ./client 22000
```

OUTPUT

```
Hello
Hello
How r u
How r u
I m fine
I m fine
Bye
Bye
```

Viva Questions:

1) In which conditions we used TCP protocol**.**
2) How can you create socket file descriptor?
3) Why we include sys/socket.h header file.
4) Why we include arpa/inet.h header file.
5) Why we use memset() function.

EXPERIMENT-8

OBJECTIVE

Write a program in C: hello_client (The server listens for, and accepts, a single UDP connection; it reads all the data itcan from that connection, and prints it to the screen; then it closes the connection)

PROGRAM

```
// UDP Client
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<string.h>
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

#define MAX_BUF 100

intmain(intargc,char*argv[])
{
intsockd;
structsockaddr_inmy_addr,srv_addr;
charbuf[MAX_BUF];
int count;
intaddrlen;

if(argc<3)
{
fprintf(stderr,"Usage: %s ip_addressport_number\n",argv[0]);
exit(1);
}
/* create a socket */
sockd=socket(AF_INET, SOCK_DGRAM,0);
if(sockd==-1)
{
perror("Socket creation error");
exit(1);
}

/* client address */
```

```
my_addr.sin_family= AF_INET;
my_addr.sin_addr.s_addr= INADDR_ANY;
my_addr.sin_port=0;

bind(sockd,(structsockaddr*)&my_addr,sizeof(my_addr));

strcpy(buf,"Hello world\n");

/* server address */
srv_addr.sin_family= AF_INET;
inet_aton(argv[1],&srv_addr.sin_addr);
srv_addr.sin_port=htons(atoi(argv[2]));

sendto(sockd,buf,strlen(buf)+1,0,
(structsockaddr*)&srv_addr,sizeof(srv_addr));

addrlen=sizeof(srv_addr);
count=recvfrom(sockd,buf, MAX_BUF,0,
(structsockaddr*)&srv_addr,&addrlen);
write(1,buf, count);

close(sockd);
return0;
```

Viva Questions:

1. Write down the syntax of recvfrom() function.
2. What down the syntax of sendto() function.
3. What is bzero function what its utility.
4. In which conditions we use UPD connection.
5. What is roundtrip time.
6.

EXPERIMENT-9

OBJECTIVE

Repeat Exercise 7 with multiplexed I/O operations.

PROGRAM

//UDP Server

```
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<string.h>
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

#define MAX_BUF 1024

intmain(intargc,char*argv[])
{
intsockd;
structsockaddr_inmy_name,cli_name;
charbuf[MAX_BUF];
int status;
intaddrlen;

if(argc<2)
{
fprintf(stderr,"Usage: %s port_number\n",argv[0]);
exit(1);
}
/* create a socket */
sockd=socket(AF_INET, SOCK_DGRAM,0);
if(sockd==-1)
{
perror("Socket creation error");
exit(1);
}

/* server address */
my_name.sin_family= AF_INET;
```

```
my_name.sin_addr.s_addr= INADDR_ANY;
my_name.sin_port=htons(atoi(argv[1]));

status=bind(sockd,(structsockaddr*)&my_name,sizeof(my_name));

addrlen=sizeof(cli_name);
status=recvfrom(sockd,buf, MAX_BUF,0,
(structsockaddr*)&cli_name,&addrlen);

printf("%s",buf);
strcat(buf,"OK!\n");

status=sendto(sockd,buf,strlen(buf)+1,0,
(structsockaddr*)&cli_name,sizeof(cli_name));

close(sockd);
return0;
}
```

Echoing back – Hello
Echoing back – How r u
Echoing back – I m fine
Echoing back – Bye

//**Client process**

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <stdio.h>
#include<string.h>
int main(intargc,char **argv)
{
intsockfd,n;
charsendline[100];
charrecvline[100];
structsockaddr_inservaddr;

sockfd=socket(AF_INET,SOCK_STREAM,0);
bzero(&servaddr,sizeofservaddr);

servaddr.sin_family=AF_INET;
servaddr.sin_port=htons(22000);

inet_pton(AF_INET,"127.0.0.1",&(servaddr.sin_addr));
```

```
connect(sockfd,(structsockaddr *)&servaddr,sizeof(servaddr));

while(1)
   {
bzero(sendline, 100);
bzero(recvline, 100);
fgets(sendline,100,stdin); /*stdin = 0 , for standard input */

write(sockfd,sendline,strlen(sendline)+1);
read(sockfd,recvline,100);
printf("%s",recvline);
   }

}
```

To Run:

```
$ gcc –o client client1.c
$ ./client 22000
```

Output:

```
Hello
Hello
How r u
How r u
I m fine
I m fine
Bye
Bye
```

Viva Questions:

1) Write down the syntax of recvfrom() function.
2) What down the syntax of sendto() function.
3) What is bzero function what its utility.
4) In which conditions we use UPD connection.
5) What is roundtrip time.

EXPERIMENT-10

OBJECTIVE

Simulate Bellman-Ford Routing algorithm in NS2.

Program:

```
#include <stdio.h>

typedefstruct {
```

```
	int u, v, w;
} Edge;

int n; /* the number of nodes */
int e; /* the number of edges */
Edge edges[1024]; /* large enough for n <= 2^5=32 */
int d[32]; /* d[i] is the minimum distance from node s to node i */

#define INFINITY 10000

voidprintDist() {
	int i;

	printf("Distances:\n");

	for (i = 0; i < n; ++i)
		printf("to %d\t", i + 1);
	printf("\n");

	for (i = 0; i < n; ++i)
		printf("%d\t", d[i]);

	printf("\n\n");
}

voidbellman_ford(int s) {
	int i, j;

	for (i = 0; i < n; ++i)
		d[i] = INFINITY;

	d[s] = 0;

	for (i = 0; i < n - 1; ++i)
		for (j = 0; j < e; ++j)
			if (d[edges[j].u] + edges[j].w < d[edges[j].v])
				d[edges[j].v] = d[edges[j].u] + edges[j].w;
}

int main(intargc, char *argv[]) {
	int i, j;
	int w;

	FILE *fin = fopen("dist.txt", "r");
	fscanf(fin, "%d", &n);
	e = 0;

	for (i = 0; i < n; ++i)
		for (j = 0; j < n; ++j) {
```

```
                        fscanf(fin, "%d", &w);
                        if (w != 0) {
                                edges[e].u = i;
                                edges[e].v = j;
                                edges[e].w = w;
                                ++e;
                        }
                }
        fclose(fin);
        /* printDist(); */
        bellman_ford(0);
        printDist();
        return 0;
}
```

OUTPUT

And here's the input file used in the example (dist.txt):

```
5
0 6 0 7 0
0 0 5 8 -4
0 -2 0 0 0
0 0 -3 9 0
2 0 7 0 0
```

Viva Questions

1) What is the time complexity of Bellman form algorithm?
2) How Bellman ford algorithm is different from Dijkastra algorithm.
3) How Bellman ford algorithm is different from Warshalls algorithm.
4) Mention the time complexity of Bellman Ford Algorithm.
5) How can you drive the complexity of Bellman Ford Algorithm.

EXPERIMENT-11

OBJECTIVE

Use of Basic Unix Shell Commands: ls, mkdir, rmdir, cd, cat, banner, touch, file, wc, sort, cut, grep, dd, dfspace, du, ulimit.

COMMAND

Basic_Linux_ Commands-

In this article you will learn most frequently used Basic Linux Commands with examples. We tried to cover as many Linux Commands as we can.

File Commands

1. The following Linux Command take you to the '/ home' directory cd /home
2. This command go back one level cd ..
3. This command takes you two folders back. cd ../..

4. This command take you to home directory cd
5. This command takes you to the user's home directory cd ~user
6. This command takes you to the previous directory cd -

"COPY" Commands in Linux

7. This command helps you copy one file to another cp file1 file2
8. Copy all files of a directory within the current work directory cp dir/* .
9. Copy a directory within the current work directory cp -a /tmp/dir1 .
10. Copy a directory cp -a dir1 dir2
11. Outputs the mime type of the file as text cp file file1
12. Linux Commands about Symlink
13. Linux Command to create a symbolic link to file or directory ln -s file1 lnk1
14. Create a physical link to file or directory ln file1 lnk1
15. View files of directory ls
16. View files of directory ls -F
17. Show details of files and directory ls -l
18. Show hidden files ls -a
19. Show files and directory containing numbers ls *[0-9]*
20. Show files and directories in a tree starting from root lstree
21. Create a directory called 'dir1' mkdir dir1
22. Create two directories simultaneously
23. Create a directory tree mkdir -p /tmp/dir1/dir2
24. Move a file or directory mv dir/file /new_path
25. Show the path of work directory pwd
26. Delete file called 'file1' rm -f file1
27. Remove a directory called 'dir1' and contents recursively rm -rf dir1
28. Remove two directories and their contents recursively rm -rf dir1 dir2
29. Delete directory called 'dir1' rmdir dir1
30. Modify timestamp of a file or directory - (YYMMDDhhmm) touch -t 0712250000 file1
31. Show files and directories in a tree starting from root(1) tree

Linux Commands for Process Management

32. The top command gives you information on the processes that currently exist. top
33. The htop command is like top, but prettier and smarter htop.
34. Use the ps command to list running processes (top and htop list all processes whether active or inactive). ps
35. A step up from the simple ps command, pstree is used to display a tree diagram of processes that also shows relationships that exist between them. pstree

36. The who command will display a list of all the users currently logged into your Linux system. who

37. As its name suggests, kill can be used to terminate a process with extreme prejudice. kill

38. The pkill and killall commands can kill a process, given its name. pkill & killall

39. pgrep returns the process IDs that match it. pgrep

40. With the help of nice command, users can set or change the priorities of processes in Linux. nice

41. It is similar to nice command. Use this command to change the priority of an already running process. renice

42. Gives the Process ID (PID) of a process pidof

43. Gives free hard disk space on your system df

44. Gives free RAM on your system free File Permissions.

Briefing about Permissions in Linux

There is a huge importance with Linux Commands when we discuss about Permissions. No restrictions on permissions. Anybody may do anything. Generally, not a desirable setting.

777 (rwxrwxrwx)

The file's owner may read, write, and execute the file. All others may read and execute the file. This setting is common for programs that are used by all users.

755 (rwxr-xr-x)

The file's owner may read, write, and execute the file. Nobody else has any rights. This setting is useful for programs that only the owner may use and must be kept private from others.

700 (rwx)

All users may read and write the file. 666 (rw-rw-rw-)

The owner may read and write a file, while all others may only read the file. A common setting for data files that everybody may read, but only the owner may change.

644 (rw-r--r--)

The owner may read and write a file. All others have no rights. A common setting for data files that the owner wants to keep private.

600 (rw)

How to use "Find Command"

The below Linux Commands gives you better Idea on find commands. You can also check more Find Commands in our other article too.

45. To find a file by name find -name "File1"

46. To find a file by name, but ignore the case of the "File1" find -iname "File1"

47. To search all files that end in ".conf" find /path -type f -name "*.conf"

48. To find all files that are exactly 50 bytes find /path -size 50c

49. To find all files less than 50 bytes find /path -size 50c

50. To find all files less than 50 bytes find /path -size -50c

51. To Find all files more than 700 Megabytes find / -size +700M
52. To find files that have a modification time of a day ago find / -mtime 1
53. To find files that were accessed in less than a day ago find / -atime -1
54. To find files that last had their meta information changed more than 3 days ago find / -ctime +3
55. To find files that were accessed in less than a minute ago find / -mmin -1
56. If we want to match an exact set of permissions find / -perm 644
57. If we want to specify anything with at least those permissions find / -perm -644

Linux Commands to check Word Count

58. Prints the number of lines in a file. wc -l file_name OR cat file_name | wc -l
59. Prints the number of words in a file. wc -w
60. Displays the count of bytes in a file. wc -c
61. Prints the count of characters from a file. wc -m
62. Prints only the length of the longest line in a file wc -L

Compression Commands (tar, tar.gz, tar.bz2 and zip Options to use the above Linux Commands

c - create a archive file. x - extract a archive file.

v- show the progress of archive file. f - filename of archive file.

t - viewing content of archive file. j - filter archive through bzip2.

z - filter archive through gzip.

r - append or update files/directories to existing archive file. w - verify a archive file.

About TAR Command

63. To Create tar Archive File

tar -cvf compress.tar /path/directory

64. To List Content of tar Archive File tar -tvf compress.tar
65. To Untar tar Archive File tar -xvf compress.tar
66. To Untar tar Archive File in a specific directory tar -xvf compress.tar -C /path/to directory
67. Untar Single file from tar File tar -xvf compress.tar file1.txt.
68. Untar Multiple files from tar tar -xvf compress.tar "file 1""file 2".
69. Extract Group of Files using Wildcard from tar Archive tar -xvf compress.tar --wildcards '*.txt'
70. To Add Files or Directories to tar Archive File tar -rvf compress.tar file/dir

About TAR.GZ

71. To Create tar.gz Archive File

tar -cvzf compresstar.gz /path/directory

72. To List Content tar.gz Archive File tar -tvf compress.tar.gz
73. To Untar tar.gz Archive File tar -zxvf compress.tar.gz
74. To Untar tar.gz Archive File in a specific directory tar -zxvf compress.tar.gz -C /path/to directory
75. Untar Single file from tar.gz File tar -zxvf compress.tar.gz file1.txt
76. Untar Multiple files from tar.gz
    tar -zxvf compress.tar.gz "file 1""file 2"
77. Extract Group of Files using Wildcard from tar.gz Archive tar -zxvf compress.tar.gz --wildcards '*.tzt'.
78. To Add Files or Directories to tar.gz tar -rvf compress.tar.gz file/dir
    About TAR.BZ2
79. To Create tar.bz2 Archive File
    tar -cvfj compress.tar.bz2 /path/directory
80. To List Content tar.bz2 Archive File tar -tvf compress.tar.bz2
81. To Uncompress tar.bz2 Archive File tar -xvf compress.tar.bz2
82. Untar Single file from tar.bz2 File tar -jxvf compress.tar.bz2 file1.txt
83. Untar Multiple files from tar.bz2
    tar -jxvf compress.tar.bz2 "file 1""file 2"
84. Extract Group of Files using Wildcard from tar.bz2 Archive tar -jxvf compress.tar.bz2 --wildcards '*.tzt'
85. To Add Files or Directories to tar.bz2 tar -rvf compress.tar.bz2 file/dir
86. To Verify tar, tar.gz and tar.bz2 Archive File tar -tvfW cmpress.tar

Linux Commands for ZIP

ZIP (The extension .zip is not mandatory and this is useful only to identify the file zip file)

87. To zipping a file or folder.

zip compress.zip file1 file2 folder1

88. To Zip individual files to a zip archive zip compress.zip file1 file2 file3
    Zipping a folder is a tricky thing as by default zip will not zip entire folder content such as sub folders and files
89. To zip first level of folder content use * as shown below zip compress.zip Folder/*
90. If there are sub folders and files in 1 folder, in order to zip all content of a folder use -r option zip -r compress.zip Folder
91. To list all the files stored in a zip file. Any of the below commands can be used and they give the same results.

unzip -l compress.zip less compress.zip zipinfo -1 compress.zip

92. To delete a file in an archive without extracting entire zip file. zip -d compress.zip path/to/file
93. To extract your files from a zip folder. unzip compress.zip
94. To extract to a specific directory use -d option unzip compress.zip -d /destination

95. To extract specific file from an archive unzip compress.zip test.sh

Linux Commands for Special Attributes on Files

96. Allows write opening of a file only append mode chattr +a file1
97. Allows that a file is compressed / decompressed automatically by the kernel chattr +c file1
98. Makes sure that the program ignores Dump the files during backup chattr +d file1
99. Makes it an immutable file, which cannot be removed, altered, renamed or linked chattr +i file1
100. Allows a file to be delete safely chattr +s file1
101. Makes sure that if a file is modified changes are written in synchronous mode as with sync chattr +S file1
102. Allows you to recover the contents of a file even if it is canceled chattr +u file1
103. Show specials attributes on file/folder lsattr file/folder

Linux Commands to know System Information

104. To know only system name, you can use uname command uname
105. To view your network hostname uname -n
106. To get information about kernel-version uname -v
107. To get the information about your kernel release uname -r
108. To get the information about your kernel release uname -r
109. To print your machine hardware name uname -m
110. All this information can be printed at once. The below two commands gives same result.
uname -a
cat /proc/version
111. Find out information about the Linux distribution and version cat /etc/*release*
112. To gather information about file system partitions fdisk -l
113. To view mounted file systems. mount
114. To view information about your CPU architecture such as number of CPU‘s, cores, CPU family model, CPU caches, threads, etc. Either of the two below commands gives same output.
lscpu
cat /proc/cpuinfo
115. To view information about block devices lsblk

Extract Information about Hardware Components using "dmidecode"

116. To print information about memory. You can get the similar output with all the below commands.
dmidecode -t memory cat /proc/meminfo
free or free -mt or free -gt
117. To print information about system dmidecode -t system
118. To print information about BIOS dmidecode -t bios
119. To print information about processor dmidecode -t processor
120. To dump all hardware information dmidecode | less

Network Commands

121. PING (Packet Internet Groper) command sends packet requests to the address you specify to

test the connectivity between 2 nodes.
ping IP/hostname
122. Ifconfig utility is used to configure network interface parameters. Mostly we use this command to check the IP address assigned to the system.
ifconfig -a
123. traceroute print the route packets take to network host. Destination host or IP is mandatory parameter to use this utility
traceroute website.com / IP
124. route command is the tool used to display or modify the routeing table. route
125. dig (Domain Information Groper) is a flexible tool for interrogating DNS name servers. It performs DNS lookups and displays the answers that are returned from the name servers.
dig website.com
126. Whois To know the information about domain like whois website.com
127. Host Command to find name to IP or IP to name host hostname
host 1.2.3.4
128. telnet connect destination host:port via a telnet protocol if connection establishes means connectivity between two hosts is working fine.
129.telnet website.com 80
130. Tracepath traces the path of the network to the destination you have provided. It attempts to list the series of hosts through which your packets travel on their way to a given destination.
tracepath website.com
131. nslookup is a program to query Internet domain name servers. nslookup website.com
132. netstat command allows you a simple way to review each of your network connections and open sockets. netstat with head output is very helpful while performing web server troubleshooting.
netstat
133. scp allows you to secure copy files to and from another host in the network. scp -r -P 22 (ssh port) user@source_hostname:/path/to/dir /destination/path
134. nmap is a very powerful command, which checks the opened port on the server. nmap hostname -p 80
SSH Commands
135. Connect to host as user ssh user@host
136. connect to host on port ssh -p port user@host

KeyBoard Shortcuts
137. Halts the current command Ctrl+C
138. Stops the current command, resume with fg in the foreground or bg in the background Ctrl+Z
139. Log out of current session, similar to exit
Ctrl+D
140. Erases one word in the current line Ctrl+W
141. Erases the whole line Ctrl+U
142. Type to bring up a recent command. You need to type the first letter of the command you are searching for.
Ctrl+R

143. Log out of current session exit.

1.Description for grep command in Unix/Linux

Grep command in Unix/Linux is a powerful tool that searches for matching a regular expression against text in a file, multiple files or a stream of input. It searches for the pattern of text that you specify on the command line and prints output for you.

In addition, three variant programs egrep,fgrep and rgrep are available.
egrep is the same as grep -E
fgrep is the same as grep -F
rgrep is the same as grep -r
Direct invocation as either egrep or fgrep is deprecated, but is provided to allow historical applications that rely on them to run unmodified.

Example of grep command in Unix/Linux: Let‘s say if you quickly want to search the string ―linux‖ in .html files on your machine? Let‘s start by searching a single file.
Here, our PATTERN string is ―linux‖ and our FILE is index.html
Search for a string ―linux‖ in a file using grep command in unix.
This is the basic usage of grep command. It searches for the given string in the specified file. grep ―linux‖ index.html.

2.Insensitive case search with grep -i

The below grep command searches for the words like ―LINUX‖, ―Linux‖, ―linux‖ case insensitively.
grep -i ―linux‖ index.html

3.Searching for a string in multiple files.

This command will search for "linux" string in multiple files at a time. It searches in all files with file1.txt, file2.txt and along with different extensions too like file1.html, file2.php and so on.
grep ―linux‖ file*.*

4.Specifying the search string as a regular expression pattern.

It is a very powerful feature and can use as a regular expression with much effectively. In the below example, it searches for all the pattern that starts with ―fast‖ and ends with ―host‖ with anything in-between. i.e To search ―fast[anything in-between]host‖ in index.html file.
grep ―fast.*host‖ index.html

5.Displaying the line numbers.

You can use this grep command to display the line number which contains the matched string in a file using the -n option
grep -n ―word*‖ file.txt

6.Highlighting the search using grep

If we use the –color option, our successful matches will be highlighted for us. grep –color ―linux‖ index.html.

7.Print the line excluding the pattern using -v option

List all the lines of the file /etc/passwd that does not contain specific word ―string‖. grep -v linux /etc/passwd.

8.Display all the lines that starts with specified pattern using ^ symbol

Bash shell treats carrot symbol (^) as a special character which treat as the beginning of line. Let‘s display the lines which starts with ―root‖ word in the file /etc/passwd.
grep ^root /etc/passwd.

9.Display all the lines that ends with specified pattern using $ symbol. List all the lines of /etc/passwd that ends with ―bash‖ word.
grep bash$ /etc/passwd

10.Search the pattern recursively using -r option

The below command will search linux in the ―/etc‖ directory recursively. grep -r linux /etc/.

11. Counting the lines when words match
This grep command can report the number of times the pattern matches for each file by using -c (count) option.
grep -c 'test' /home/example/test.txt

10 df Commands to Check Space in Linux or Ubuntu

df Command in Linux
In Linux, you can check disk space using command line tool called df command. The df command stands for Disk File System. Using df command you can find disk space summary information like available and used disk space on Linux.
In this tutorial, we are going to teach you ten different df commands with practical examples to manage disk space on Linux.

Let‘s explore df command with all the options so that you will get complete idea on Linux disk space.

The basic syntax for df command is:

df [options] [devices]

1.Checking File System Disk Space

The ―df‖ command displays the information of file system device names, disk blocks, total disk space used, available disk space, percentage of usage and mount points on a file system.
You could see a similar output on your screen. root@fwh:~# df

```
Filesystem  1K-blocks      Used Available Use% Mounted on

/dev/sda1   10179896 3579164 6060584 38% /

none        4        0       4 0% /sys/fs/cgroup
udev        1020812          4 1020808 1% /dev
tmpfs       205016           336     204680 1% /run
none        5120             0       5120 0% /run/lock
none        1025072          0 1025072 0% /run/shm
none        102400           0       102400 0% /run/user
```

2. Display Disk Space in Human Readable Format

As you see in the first example, the output is difficult to read or understand. By default df command shows disk space information in bytes which is difficult to understand for humans. We can easily understand if the results are in megabytes and gigabytes.
Don‘t worry, the good df command has an option to display information in human readable format like in MB and GB. You just need to add -h flag to see.
root@fwh:~# df -h

```
Filesystem  Size    Used Avail Use% Mounted on
/dev/sda1   9.8G    3.5G 5.8G 38% /
none        4.0K    0 4.0K 0% /sys/fs/cgroup
udev        997M    4.0K 997M 1% /dev
tmpfs       201M    336K 200M 1% /run
none        5.0M    0 5.0M 0% /run/lock
none        1002M   0 1002M 0% /run/shm
none        100M    0 100M 0% /run/user
```

3.Display information of all file systems

If you want to see all file systems including which has zero block sizes, pass parameter -a or -all along with df command. The output would be similar to this.

```
root@fwh:~# df -a
Filesystem          1K-blocks      Used Available Use% Mounted on
/dev/sda1  10179896 3579176 6060572 38% /

proc        0         0         0         - /proc
sysfs       0         0         0         - /sys

none        4         0         4 0% /sys/fs/cgroup

none        0         0         0         - /sys/fs/fuse/connections
none        0         0         0         - /sys/kernel/debug
none        0         0         0         - /sys/kernel/security

udev
1020812
4 1020808 1% /dev
devpts      0         0         0         - /dev/pts
tmpfs       205016          336       204680 1% /run
none        5120    0         5120 0% /run/lock
none        1025072         0 1025072 0% /run/shm
none        102400          0         102400 0% /run/user
none        0         0         0         - /sys/fs/pstore
systemd     0         0         0         - /sys/fs/cgroup/systemd
```

4.Display File System details in Bytes

To display all file system information and usage in 1 K blocks , pass the option _-k_ as follows.

```
root@fwh:~# df -k
Filesystem          1K-blocks      Used Available Use% Mounted on

/dev/sda1  10179896 3579188 6060560 38% /

none        4         0         4 0% /sys/fs/cgroup
udev        1020812             4 1020808 1% /dev
tmpfs       205016              336       204680 1% /run
none        5120          0         5120 0% /run/lock
```

```
none        1025072          0 1025072 0% /run/shm
none        102400           0    102400 0% /run/user
```

5.Display File System Information in MB

To display all file system information and usage in MB or megabytes , pass the option _-m_. root@fwh:~# df -m

```
Filesystem      1M-blocks Used Available Use% Mounted on

/dev/sda1  9942  3496     5919 38% /

none        1        0        1 0% /sys/fs/cgroup
udev        997      1        997 1% /dev
tmpfs       201      1        200 1% /run
none        5        0        5 0% /run/lock
none        1002     0        1002 0% /run/shm
none        100      0        100 0% /run/user
```

6.Display File System Information in GB

To display all file system details and usage in GB or gigabytes , pass the option _-h_. root@fwh:~# df -h

```
Filesystem      Size Used Avail Use% Mounted on
/dev/sda1  9.8G 3.5G 5.8G 38% /
none        4.0K    0 4.0K 0% /sys/fs/cgroup

udev 997M 4.0K 997M 1% /dev tmpfs   201M 336K 200M 1% /run none        5.0M    0    5.0M    0%
/run/lock
none 1002M 0 1002M 0% /run/shm
none 100M 0 100M 0% /run/user
```

7.Display /home file system information

You can see only /home file system device information by executing below df command. root@fwh:~# df -h /home

```
Filesystem      Size Used Avail Use% Mounted on

/dev/sda1 9.8G 3.5G 5.8G 38% /
```

8. Display File System Type in Linux

To see the type of each file system, pass the -T option. It gives output with file system types column. You could see the type of each file system listed such as ext4, ext3, tempfs, devtmpfs and etc.

root@fwh:~# df -T

Filesystem Type 1K-blocks Used Available Use% Mounted on /dev/sda1
ext4 10179896 3579192 6060556 38% /

none tmpfs 4 0 4 0% /sys/fs/cgroup
udev devtmpfs 1020812 4 1020808 1% /dev
tmpfs tmpfs 205016 336 204680 1% /run
none tmpfs 5120 0 5120 0% /run/lock
none tmpfs 1025072 0 1025072 0% /run/shm
none tmpfs 102400 0 102400 0% /run/user

9. Include or Exclude only certain File System Types.

If you want to see only ext4 file systems, use df command with option -t root@fwh:~# df -t ext4 .
Filesystem 1K-blocks Used Available Use% Mounted on /dev/sda1 10179896 3579280 6060468 38% /

If you want to see all file system types except ext4, then pass -x option and type (ext3, ext4) to exclude from the output.

root@fwh:~# df -x ext4

Filesystem 1K-blocks Used Available Use% Mounted on none 4 0 4 0% /sys/fs/cgroup
udev 1020812 4 1020808 1% /dev
tmpfs 205016 336 204680 1% /run
none 5120 0 5120 0% /run/lock
none 1025072 0 1025072 0% /run/shm
none 102400 0 102400 0% /run/user

10. Display df Command Options and Help

You can see all the available options with df command by typing below command. root@fwh:~# df --help
Usage: df [OPTION]... [FILE]...
Show information about the file system on which each FILE resides, or all file systems by default.
Mandatory arguments to long options are mandatory for short options too.
-a, --all include dummy file systems
-B, --block-size=SIZE scale sizes by SIZE before printing them. E.g., '-BM' prints sizes in units of 1,048,576 bytes.
See SIZE format below.

--total produce a grand total
-h, --human-readable print sizes in human readable format (e.g., 1K 234M 2G)
-H, --si likewise, but use powers of 1000 not 1024
-i, --inodes list inode information instead of block usage
-k like --block-size=1K
-l, --local limit listing to local file systems
--no-sync do not invoke sync before getting usage info (default)
--output[=FIELD_LIST] use the output format defined by FIELD_LIST, or print all fields if FIELD_LIST is omitted.
-P, --portability use the POSIX output format
--sync invoke sync before getting usage info
-t, --type=TYPE limit listing to file systems of type TYPE
-T, --print-type print file system type
-x, --exclude-type=TYPE limit listing to file systems not of type TYPE
-v (ignored)
--help display this help and exit

--version output version information and exit.

Viva Questions

1. What is the basic difference between BASH and DOS?
2. BASH commands are case sensitive while DOS commands are not?
3. DOS follows a convention in naming files, which is 8 character file name followed by a dot and characters for the extension?
4. What is the importance of the GNU project?
5. Describe the root account.
6. What is CLI?
7. What is GUI?
8. How do you open a command prompt when issuing a command?

EXPERIMENT-12

OBJECTIVE

Commands related to inode, I/O redirection and piping, process control commands, mails.

COOMAND

Redirection Summary

Redirection Operator	Resulting Operation

command > file	stdout written to file, overwriting if file exists
command >> file	stdout written to file, appending if file exists
command < file	input read from file
command 2> file	stderr written to file, overwriting if file exsits
command 2>> file	stderr written to file, appending if file exists
command > file 2>&1	stdout written to file, stderr written to same file descriptor

As an Operating System which inspired from Unix, Linux has a built-in tool to capture current processes on the system. From 'ps' manual page, it gives a snapshot of the current processes. It will capture the system condition at a single time. If you want to have repetitive updates in a real- time, we can use top command.

PS support three (3) type of usage syntax style.

1.UNIX style, which may be grouped and must be preceded by a dash

2.BSD style, which may be grouped and must not be used with a dash

3.GNU long options, which are preceded by two dash

We can mix those style, but conflicts can appear. In this article, will use UNIX style. Here ‘s are some examples of PS command in a daily use.

1.Run ps without any options

This is a very basic ps usage. Just type ps on your console to see its result. ps with no options
By default, it will show us 4 columns of information. PID is a Process ID of the running command (CMD).

TTY is a place where the running command runs
TIME tell about how much time is used by CPU while running the command CMD is a command that runs as a current process. This information is displayed in an unsorted result.

2.Show all current processes

To do this, we can use -a options. As we can guess, -a is stand for ―all While x will show all process even the current process is not associated with any TTY (terminal)
$ ps -ax

This result might be a long result. To make it easier to read, combine it with less command.

$ ps -ax | less

ps all information

3.Filter processes by its user

For some situation, we may want to filter processes by a user. To do this, we can use -u option. Let

say we want to see what processes which run by user pungki. So the command will be like below
$ ps -u pungki filter by user.

4.Filter processes by CPU or memory usage

Another thing that you might want to see is to filter the result by CPU or memory usage. With this, you can grab information about which processes that consume your resource. To do this, we can use aux options. Here‘s an example of it :
$ ps -aux | less

show all information

Since the result can be in a long list, we can pipe less command into ps command. Explaining each terms in the result.

1.USER - Username: The name of the user associated with the process.

2.PID - Process ID: The unique numeric identifier assigned to the process.

3.%CPU - Percentage of CPU: Time used (total CPU time divided by length of time the process has been running).

4.%MEM - Percentage of RAM Memory: used (memory used divided by total memory available).

5. VSZ - Virtual Memory Size: Size of the process in virtual memory expressed in KiB.

6. RSS - Resident Set Size.

7. TTY - Terminal controlling the process.

8. STAT - Process State - Possible values.

* R - Running.

* S - Sleeping (may be interrupted).

* D - Sleeping (may not be interrupted) - used to indicate process is handling input/output.

* T - Stopped or being traced.

* Z - Zombie or "hung" process.

9.START: The date or time at which the process started.

10.TIME: Cumulative CPU time used by the process and the child processes started by the process.

11.COMMAND: Command used to start the process.

By default, the result will be in unsorted form. If we want to sort by particular column, we can add

--sort option into ps command.
Sort by the highest CPU utilization in ascending order

$ ps -aux --sort -pcpu | less sort by cpu usage
Sort by the highest Memory utilization in ascending order

$ ps -aux --sort -pmem | less sort by memory usage
Or we can combine it into a single command and display only the top ten of the result :

$ ps -aux --sort -pcpu,+pmem | head -n 10
5.Top 10 memory consuming processes

This shows the top 10 memory consuming processes running on the system. It is useful to see what processes are the most memory consuming.
Options:

--sort spec specifies sorting order. Sorting syntax is [+|-]key[,[+|-]key[,...]]. Choose a multi-letter key from the STANDARD FORMAT SPECIFIERS section. The "+" is optional since default direction is increasing numerical or lexicographic order. Identical to k. For example: ps jax --sort=uid,-ppid,+pid

ps -auxf | sort -nr -k 4 | head -10

root 3025 3.2 13.0 1600004 245276 ? Sl 10:00 2:01 _ /usr/bin/gnome-shell

root 2261 0.5 1.8 188108 34540 tty1 Rs+ 09:49 0:25 _ /usr/bin/Xorg :0 -background none -verbose -auth /run/gdm/auth-for-gdm-fTksZM/database -seat seat0 -nolisten tcp vt1
root 3089 0.0 1.2 1060728 22608 ? Sl 10:00 0:00 /usr/bin/nautilus --no-default-window

root 2957 0.0 1.2 998384 23244 ? Sl 10:00 0:00 _ /usr/libexec/gnome-settings-daemon

root 3482 0.0 1.0 632688 19344 ? Rl 10:07 0:02 /usr/libexec/gnome-terminal-server

root 3181 0.1 1.0 349960 20280 ? S 10:00 0:07 /usr/bin/vmtoolsd -n vmusr

root 802 0.0 0.9 550008 18036 ? Ssl 09:49 0:01 /usr/bin/python -Es /usr/sbin/tuned -l -P

root 950 0.0 0.8 102312 15580 ? S 09:49 0:00 _ /sbin/dhclient -d -sf /usr/libexec/nm-dhcp- helper -pf /var/run/dhclient-eno16777736.pid -lf /var/lib/NetworkManager/dhclient-42456ebc- a752-4950-9adf-1b8bc29f43eb-eno16777736.lease -cf /var/lib/NetworkManager/dhclient-eno16777736.

conf eno16777736 root 3154 0.0 0.8 898940 15300 ? Sl 10:00 0:00 /usr/libexec/evolution-calendar-factory.

root 3020 0.0 0.8 551480 16420 ? Sl 10:00 0:00 /usr/libexec/goa-daemon ps memory.

6.Filter processes by its name or process ID

To do this, we can use -C option followed by the keyword. Let say, we want to show processes named getty. We can type :
$ ps -C getty

filter by its name or process ID

If we want to show more detail about the result, we can add -f option to show it on full format listing. The above command will looks like below :
$ ps -f -C getty
filter by its name or process ID

7.Filter processes by a thread of the process

If we need to know the thread of a particular process, we can use -L option followed by its Process ID (PID). Here‘s an example of -L option in action :
$ ps -L 1213

show processes in threaded view

As we can see, the PID remain the same value, but the LWP which shows numbers of thread show different values.

8.Displays All threads of a specific process id

This will show all the threads of a particular process pid. Options:
-L show threads possibly with LWP and NLWP columns. # ps -Lf -p 3482
UID PID PPID LWP C NLWP STIME TTY TIME CMD

root 3482 1 3482 0 4 10:07 ? 00:00:03 /usr/libexec/gnome-terminal-server

root 3482 1 3483 0 4 10:07 ? 00:00:00 /usr/libexec/gnome-terminal-server

root 3482 1 3484 0 4 10:07 ? 00:00:00 /usr/libexec/gnome-terminal-server

root 3482 1 3487 0 4 10:07 ? 00:00:00 /usr/libexec/gnome-terminal-server ps thread

9.Shows child of a parent process

This will display all child processes of a process and is useful in finding out what processes have been forked out of this main process.
ps -o pid,pcpu,pmem,uname,comm -C apache2 PID %CPU %MEM USER COMMAND
2642 0.0 3.4 www-data apache2
4185 0.0 3.6 www-data apache2

4186 0.0 3.3 www-data apache2

4187 0.0 3.3 www-data apache2

5359 0.0 3.3 www-data apache2

13343 0.0 3.5 www-data apache2

17228 0.0 3.5 www-data apache2

21037 0.0 3.4 www-data apache2 ps child

10.Show processes in the hierarchy

Sometimes we want to see the processes in hierarchical form. To do this, we can use -axjf options.
$ps -axjf

show in hierarchy

Or, another command which we can use is.pstree

$ pstree

show information in hierarchy

11.Displays a Process Duration

This will show for how long a process has run on the system. # ps -e -o pid,comm,etime | grep mysql
3107 mysqld_safe 18-07:01:53

3469 mysqld 18-07:01:52

We can even use these following two keywords to find the uptime of a live process. etime: elapsed time since the process was started, in the form [[DD-]hh:]mm:ss. etimes: elapsed time since the process was started, in seconds.

First, you need to find out the PID of a process. The following command displays the PID of apache2.
pidof apache2 21774
Now, we can find how long this process has been running using the command: # ps -p 21774 -o etime
ELAPSED 2-23:31:39
You can also view the elapsed time in seconds using etimes keyword as below: # ps -p 21774 -o etimes
ELAPSED 257895

12.Show security information

If we want to see who is currently logged on into your server, we can see it using the ps command. There are some options that we can use to fulfill our needs. Here are some examples:
$ ps -eo pid,user,args

Option -e will show you all processes while -o option will control the output. Pid, User and Args will show you the Process ID, the User who run the application and the running application.
show security information

The keyword / user-defined format that can be used with -e option are args, cmd, comm, command, fname, ucmd, ucomm, lstart, bsdstart and start.

13.Show every process running as root (real & effective ID) in user format

System admin may want to see what processes are being run by root and other information related to it. Using ps command, we can do by this simple command :
$ ps -U root -u root u

The -U parameter will select by real user ID (RUID). It selects the processes whose real user name or ID is in the userlist list. The real User ID identifies the user who created the process.
While the -u paramater will select by effective user ID (EUID)

The last u paramater, will display the output in user-oriented format which contains User, PID, %CPU, %MEM, VSZ, RSS, TTY, STAT, START, TIME and COMMAND columns.

Here‘s the output of the above command. show real and effective User ID.

14.Use PS in a realtime process viewer

ps will display a report of what happens in your system. The result will be a static report.

Let say, we want to filter processes by CPU and Memory usage as on the point 4 above. And we want the report is updated every 1 second. We can do it by combining ps command with watch command on Linux.
Here‘s the command :
$ watch -n 1 _ps -aux --sort -pmem, -pcpu‘ combine ps with watch
If you feel the report is too long, we can limit it by - let say - the top 20 processes. We can add head command to do it.

$ watch -n 1 _ps -aux --sort -pmem, -pcpu | head 20‘ combine ps with watch.

This live reporter is not like top or htop of course. But the advantage of using ps to make live report is that you can custom the field. You can choose which field you want to see.

For example, if you need only the pungki user shown, then you can change the command to become like this :
$ watch -n 1 _ps -aux -U pungki u --sort -pmem, -pcpu | head 20‘ combine ps with watch

For documentation, you can type man ps on your Linux console to explore more options.

MAIL COMMAND

Send mails from command-line

Being able to send emails from command-line from a server is quite useful when you need to generate emails programmatically from shell scripts or web applications for example.

This tutorial explains, how to use to the mail command on linux to send mails from the command-line using the mail command.
How the mail command works

For those who are curious about how exactly the mail command delivers the mails to the recipients, here is a little quick explanation.

The mail command invokes the standard sendmail binary (/usr/sbin/sendmail) which in turns connects to the local MTA to send the mail to its destination. The local MTA is a locally running smtp server that accepts mails on port 25.
mail command -> /usr/sbin/sendmail -> local MTA (smtp server) -> recipient MTA (and Inbox)

This means that an smtp server like Postfix should be running on the machine where you intend to use the mail command. If none is running you get the error message "send-mail: Cannot open mail:25".
Install the mail command
The mail command is available from many different packages. Here is the list -

1. gnu mailutils

2. heirloom-mailx

3. bsd-mailx

Each flavor has a different set of options and supported features. For example the mail/mailx command from the heirloom-mailx package is capable of using an external smtp server to send messages, while the other two can use only a local smtp server.

In this tutorial we shall be using the mail command from the mailutils package, which is available on most Debian and Ubuntu based systems.

Use the apt-get command to install it

$ apt-get install mailutils

Now you should have the mail command ready to work.

1.Use the mail command

Run the command below, to send an email to someone@example.com. The s option specifies the subject of the mail followed by the recipient email address.

$ mail -s "Hello World" someone@example.com

The above command is not finished upon hitting Enter. Next you have to type in the message. When you're done, hit 'Ctrl-D' at the beginning of a line

$ mail -s "Hello World" someone@example.com Cc:
Hi Peter How are you I am fine Good Bye

<Ctrl+D>
The shell asks for the 'Cc' (Carbon copy) field. Enter the CC address and press enter or press enter without anything to skip.
From the next line type in your message. Pressing enter would create a new line in the message. Once you are done entering the message, press . Once you do that, the mail command would dispatch the message for delivery and done.

2.Subject and Message in a single line

To specify the message body in just one line of command use the following style

$ mail -s "This is the subject" somebody@example.com <<<'This is the message' Or like this

$echo "This is the body" | mail -s "Subject" -aFrom:Harry\<harry@gmail.com\> someone@example.com

3.Take message from a file

If the email message is in a file then we can use it directly to send the mail. This is useful when calling the mail command from shell scripts or other programs written in perl or php for example.
$ mail -s "Hello World" user@yourmaildomain.com < /home/user/mailcontent.txt Or a quick one liner
$ echo "This is the message body" | mail -s "This is the subject" mail@example.com

4.Specify CC and BCC recipients

Other useful parameters in the mail command are:

-c email-address (CC - send a carbon copy to email-address)

-b email-address (BCC - send a blind carbon copy to email-address) Here's an example of how you might use these options
$ mail -s "Hello World" user1@example.com -c usertocc@example.com -b usertobcc@example.com.

5.Sending to multiple recipients

It is also possible to specify multiple recipients by joining them with a comma.

$ mail -s "Hello World" user1@example.com,user2@example.com

6.Specify the FROM name and address

The "-a" option allows to specify additional header information to attach with the message. It can be used to provide the "FROM" name and address. Here is a quick example

echo "This is the message body" | mail -s "This is the subject" mail@example.com -aFrom:sender@example.com
The a option basically adds additional headers. To specify the from name, use the following syntax.
$ echo "This is the body" | mail -s "Subject" -aFrom:Harry\<harry@gmail.com\> someone@example.com

Note that we have to escape the less/great arrows since they have special meaning for the shell prompt. When you are issuing the command from within some script, you would omit that.

7.Send mail to a local system user

To send mail to a local system user just use the username in place of the recipient address

$ mail -s "Hello World" username

You could also append "@hostname" to the username, where the hostname should be the hostname of the current system.

8.Verbose output

Sometimes when testing mail servers, you would want to check the SMTP commands being used by the mail command. Use the "-v" option for that
$ mail -v -s "This is the subject" somebody@example.com <<<'This is the message'

If the mail fails to deliver due to an improperly configured mail server for example, the smtp command log will show what has gone wrong.
Send mail with attachments using Mutt

The mail command could do some basic things till now, but moving forward, it lacks important features like sending attachments.

So we have to use another command line tool called mutt. Mutt is like an enhanced version of the mail command with a very similar syntax.

Debian / Ubuntu users can install mutt with the apt command.
$ apt-get install mutt

Fedora / CentOS or Red Hat Enterprise Linux (RHEL) users:

$ yum install mutt

Now you are ready to send mail with attachments with command line interface.

Send a simple mail .

$ echo "This is mutt from universe" | mutt -s "This is mutts subject" someone@example.com Send mail with attachment.

Use the "a" option to specify the path of the file to attach

$ mutt -s "Subject" -a /path/to/file -- user@example.com < home/user/mailcontent.txt

According to the syntax of mutt options, it is necessary to separate the files and the recipients with a double dash "--". Also the "-a" option should be last one.
Send mail with bash/shell scripts

This example demonstrates how the output of a command can be used as the message in the email.

Here is an easy shell script that reports disc usage over mail. #!/bin/bash
du -sh | mail -s "disk usage report" user@yourmaildomain.com

Open a new file and add the lines above to that file, save it and run on your box. You will receive an email that contains "du -sh" output.

Read mails

This is not something interesting and you would not be doing this in a real life scenario. It is just being shown for the sake of it.

The mail command can be used to read mails. Just run it without an options and it would list all the mails in your inbox
$ mail

Here's a sample output

$ mail

Heirloom mailx version 12.5 6/20/10. Type ? for help. "/var/mail/enlightened": 7 messages 3 unread
O 1 Enlightened Sat Dec 6 11:3321/658 This is the subject
O 2 Enlightened Sat Dec 6 11:34773/25549 This is the subject
O 3 Enlightened Sat Dec 6 16:43 20/633 This is the subject
O 4 Enlightened Sat Dec 6 16:44 20/633 This is the subject

U 5 Mail Delivery Syst Sat Dec 6 16:50 74/2425 Undelivered Mail Returned to Sender U 6 Enlightened Sat Dec 6 16:51 19/632 This is mutts subject

U 7 Enlightened Sat Dec 6 16:52 19/647 This is mutts subject

?

At the end is q question mark which is an interactive prompt waiting for your command. Simply enter the number of the email you want to read and hit enter. It would open up the mail then.
After you are done reading the email, enter 'q' and hit enter to come back. Enter z and hit enter to bring back the list of emails.

The mail command by default reads the emails from the directory "/var/mail/". So every user has a separate mail directory. This way of storing and fetching mails is not very useful or practical in real life, where mail address consist of domain name along with username and a single server could be hosting emails for multiple domains.

Maildir-utils command

'mu' is a set of command-line tools for Linux/Unix that enable you to quickly find the e-mails you are looking for.

Debian/Ubuntu users can use the apt-get command to install it # apt-get install maildir-utils
To search mails from william with subject report use the following command -

$ mu find from: William subject: report

To check the current mail configurations, use the info option. # mu-tool info
VERSION=2.99.97 SYSCONFDIR=/etc

MAILSPOOLDIR=/var/mail/ SCHEME=mbox LOG_FACILITY=mail

Viva Questions

1. How do you open a command prompt when issuing a command?
2. How can you find out how much memory Linux is using?
3. What is a typical size for a swap partition under a Linux system?
4. What are symbolic links?
5. Does the Ctrl+Alt+Del key combination work on Linux?
6. How do you refer to the parallel port where devices such as printers are connected?
7. Are drives such as hard drive and floppy drives represented with drive letters?
8. How do you change permissions under Linux?
9. In Linux, what names are assigned to the different serial ports?

10. How do you access partitions under Linux?
11. What are hard links?
12. What is the maximum length for a filename under Linux?
13. What are filenames that are preceded by a dot?
14. Explain virtual desktop.
15. How do you share a program across different virtual desktops under Linux?

EXPERIMENT-13

OBJECTIVE

Shell Programming: Shell script based on control structure- If-then-fi, if-then else-if, nested if-else, to find:

(A) Greatest among three numbers.

(B) To find a year is leap year or not.

(C) To calculate profit or loss.

(D) To input angles of a triangle and find out whether it is valid triangle or not.

(E) To check whether a character is alphabet, digit or special character.

(A) Greatest among three numbers.

PROGRAM

```
#!/bin/bash
echo "enter first number" read first
echo "enter second number" read sec
echo "enter third number" read third
if [ $first -gt $sec ] ; then if [ $first -gt $third ] ; then
echo -e " $first is greatest number " else
echo -e " $third is greatest number " fi
else
if [ $sec -gt $third ] ; then
echo -e " $sec is greatest number " else
echo -e " $third is greatest number " fi
fi
```

OUTPUT

```
$ gedit great.sh
$ bash great.sh enter first number
34
 enter second number
 78
```

enter third number
10
78 is greatest number

(B) To find a year is leap year or not.

PROGRAM

```
#!/bin/bash
echo -e "enter year " read year
#if year is less than or equal to 0 then current year is assigned to year variable if [ $year -le 0 ] ; then
year=`date | cut -d "" -f6` fi
if [ `expr $year % 400` -eq 0 ]; then echo "$year is a Leap year"
elif [ `expr $year % 100` -eq 0 ]; then echo "$year is not a Leap year"
elif [ `expr $year % 4 ` -eq 0 ]; then echo "$year is not a Leap year" else
echo "$year is not a leap year" fi
```

OUTPUT

```
gedit leap.sh
bash leap.sh enter year
2006
2006 is not a leap year
bash leap.sh enter year
2000
2000 a Leap year
```

(C) To calculate profit or loss.

PROGRAM

```
echo -e "Enter Cost Price:\c" read cp
echo -e "Enter Selling Price:\c" read sp
if [ $sp -eq $cp ]; then
echo -e "\nNo profit or loss has incurred.\n" elif [ $sp -lt $cp ];
then
```

```
echo -e "\nLoss of Rs.`expr $cp - $sp` has incurred.\n" else
echo -e "\nProfit of Rs.`expr $sp - $cp` has incurred.\n" fi
```

OUTPUT-

```
gedit pl1.sh
bash pl1.sh
Enter Cost Price:1000
Enter Selling Price:1250
Profit of Rs.250 has incurred.
```

(D) To input angles of a triangle and find out whether it is valid triangle or not.

PROGRAM

```
# Taking input from user
echo "Enter the angles of triangle"
read A1
read A2
read A3
# storing the sum of angles in sum variable
sum=$((A1+A2+A3))
# checking if sum is equal to 180
if [ $sum -eq 180 ]
# if condition is satisfied
 then
 echo "Valid triangle"
# if condition is not satisfied
else
echo "Invalid triangle"
# end of if loop
  fi
```

OUTPUT

```
Enter the angles of triangle
80
```

60
40
Valid triangle

(E) To check whether a character is alphabet, digit or special character.

PROGRAM

```
echo "enter a char"
read c

#set specified is [a-z],

which is a shorthand way of typing [abcdefghijklmnopqrstuvwxyz]

#set specified is [0-9], which is a shorthand way of typing [0123456789]
if [[ $c == [A-Z] ]];
then
   echo "you have entered an alphabet"
elif [[ $c == [a-z] ]];
then
   echo "you have entered an alphabet"
elif [[ $c == [0-9] ]];
then
   echo "you have entered is a digit"
else
   echo "you have entered a special symbols!"
fi
```

OUTPUT

```
[cloudera@quickstart Desktop]$ sh ads.sh
enter a char
A
you have entered an alphabet
[cloudera@quickstart Desktop]$ sh ads.sh
enter a char
a
you have entered an alphabet
[cloudera@quickstart Desktop]$ sh ads.sh
enter a char
1
you have entered is a digit
[cloudera@quickstart Desktop]$ sh ads.sh
enter a char
!
you have entered a special symbols!
```

Viva Questions

1) What is command grouping and how does it work?

2) How do you execute more than one command or program from a single command line entry?

3) Write a command that will look for files with an extension "c", and has the occurrence of the string "apple" in it.

4) Write a command that will display all .txt files, including its individualpermission. ls -al

5) Write a command that will do the following:

6) What, if anything, is wrong with each of the following commands?

a) ls -l-s

b) cat file1, file2

c) ls - s Factdir

7) What is the command to calculate the size of a folder?

8) How can you find the status of a process?

8) How can you check the memory status?

9) Explain how to color the Git console?

10)How can you append one file to another in Linux?

11)Explain how you can find a file using Terminal?

12)Explain how you can create a folder using Terminal?

13)Explain how you can view the text file using Terminal?

14)Explain how to enable curl on Ubuntu LAMP stack?

15)Explain how to enable root logging in Ubuntu?

EXPERIMENT-14

OBJECTIVE

Write a shell script to print different shapes- Diamond, triangle, square, rectangle, hollow

square etc.

PROGRAM

Rectangle-

```
/*
*   C program to print rectangle star pattern
*/
#include <stdio.h> int main()
{
int i, j, rows, columns;
/* Input rows and columns from user */ printf("Enter number of rows: ");
scanf("%d", &rows);
printf("Enter number of columns: ");
scanf("%d", &columns);
/* Iterate through each row */ for(i=1; i<=rows; i++)
{
/* Iterate through each column */ for(j=1; j<=columns; j++)
{
/* For each column print star */ printf("*");
}
/* Move to the next line/row */ printf("\n");
}

return 0;
}
```

OUTPUT

```
$ gedit rectangle.c
$ gcc -o rectangle rectangle.c
$ ./rectangle

Enter number of rows: 5 Enter number of columns: 10
**********
**********
**********
**********
**********
```

DIAMOND

PROGRAM

```
/*
*C program to print diamond star pattern
*/
#include <stdio.h> int main()
{
int i, j, rows;
int stars, spaces;
printf("Enter rows to print : "); scanf("%d", &rows);
stars = 1;
spaces = rows - 1;
/* Iterate through rows */ for(i=1; i<rows*2; i++)
{
/* Print spaces */ for(j=1; j<=spaces; j++) printf("");
/* Print stars */ for(j=1; j<stars*2; j++) printf("*");
/* Move to next line */ printf("\n");
if(i<rows)
{
spaces--; stars++;
}
else
```

```
{
spaces++; stars--;
}
}
return 0;
}
```

OUTPUT

```
$ gedit diamond.c
$ gcc -o diamond diamond.c
$ ./diamond

Enter rows to print : 5
*
***
*****
*******
*********
*******
*****
***
*
```

TRIANGLE-

PROGRAM

```
/*
* C program to print right triangle star pattern series
*/
#include <stdio.h> int main()
{
int i, j, n;
```

```
/* Input number of rows from user */
printf("Enter value of n: ");
scanf("%d", &n);
for(i=1; i<=n; i++)
{
/* Print i number of stars */
for(j=1; j<=i; j++)
{
printf("*");
}
/* Move to next line */ printf("\n");
}
return 0;
}
```

OUTPUT

```
$ gedit triangle.c
$ gcc -o triangle triangle.c
$ ./triangle


Enter value of n: 5
*
**
***
****
*****
```

SQUARE

PROGRAM

```
/*
```

```
*C program to print square star pattern
*/
#include <stdio.h> int main()
{
int i, j, N;
/* Input number of rows from user */
printf("Enter number of rows: ");
scanf("%d", &N);
/* Iterate through N rows */ for(i=1; i<=N; i++)
{
/* Iterate over columns */ for(j=1; j<=N; j++)
{
/* Print star for each column */ printf("*");
}
/* Move to the next line/row */ printf("\n");
}
return 0;
}
```

OUTPUT

```
$ gedit sqr.c
$ gcc -o sqr sqr.c
$ ./sqr

Enter number of rows: 5
*****
*****
*****
*****
****
```

HOLLOW SQUARE

PROGRAM

```
/**
* C program to print hollow square star pattern
*/
#include <stdio.h> int main()
{
int i, j, N;
/* Input number of rows from user */ printf("Enter number of rows: ");
scanf("%d", &N);
/* Iterate over each row */ for(i=1; i<=N; i++)
{
 /* Iterate over each column */ for(j=1; j<=N; j++)
{
if(i==1 || i==N || j==1 || j==N)
{
/* Print star for 1st, Nth row and column */ printf("*");
}
else
{
printf("");
}
}
/* Move to the next line/row */ printf("\n");
}
return 0;
}
```

OUTPUT

```
$ gedit hsqr.c
$ gcc hsqr hsqr.c gcc: error: hsqr: No such file or directory
$ gcc -o hsqr hsqr.c
./hsqr
Enter number of rows: 5
*****
```

```
*   *
*   *
*   *
*****
```

Viva Questions

1) What is the alternative command available to echo and what does it do?

2)How to find out the number of arguments passed to the script?

3) What are control instructions and how many types of control instructions are available in a shell? Explain in brief.

4)What are Loops and explain three different methods of loops in brief?

5) What is IFS?

6)What is a Break statement and what is it used for?

7)What is Continue statement and what is it used for?

8)What are Meta characters in a shell? Explain with some examples.

9)How to execute multiple scripts? Explain with an example.

10) Which command needs to be used to know how long the system has been running?

EXPERIMENT-15

OBJECTIVE

Shell Programming- Arrays

(A) WAP in C to read and print elements of array.
(B) WAP in C to find sum of all array elements.
(C) WAP in C to find reverse of array elements.
(D) WAP in C program to search an element in an array.
(E) WAP in C program to sort array elements in ascending or descending order.

(A) WAP in C to read and print elements of array.

PROGRAM

```
#include <stdio.h>
int main()
{
int arr[10];
int i;
printf("\n\nRead and Print elements of an array:\n");
printf("     \n");
printf("Input 10 elements in the array :\n");
for(i=0; i<10; i++)
{
printf("element - %d : ",i);
scanf("%d", &arr[i]);
}

printf("\nElements in array are: ");
for(i=0; i<10; i++)
{

printf("%d ", arr[i]);

}

printf("\n");

}
```

OUTPUT

$ gedit arr1.c

```
$ gcc -o arr1 arr1.c

$ ./arr1
```

Read and Print elements of an array:

```
Input 10 elements in the array : element - 0 : 9
element - 1 : 4
element - 2 : 6
element - 3 : 3
element - 4 : 4
element - 5 : 56
element - 6 : 23
element - 7 : 65
element - 8 : 78
element - 9 : 90

Elements in array are: 9 4 6 3 4 56 23 65 78 90
```

(B) WAP in C to find sum of all array elements.

PROGRAM

```
#include <stdio.h>
int main()
{
int a[100];
int i, n, sum=0;
printf("\n\nFind sum of all elements of array:\n");
printf("     \n");
printf("Input the number of elements to be stored in the array :");
scanf("%d",&n);
printf("Input %d elements in the array :\n",n);
for(i=0;i<n;i++)
{
printf("element - %d : ",i);
scanf("%d",&a[i]);
}
for(i=0; i<n; i++)
{
sum += a[i];
}
printf("Sum of all elements stored in the array is : %d\n\n", sum);
}
```

OUTPUT

$ gedit sum1.c

$ gcc -o sum1 sum1.c

$ ./sum1

Find sum of all elements of array:

Input the number of elements to be stored in the array :4 Input 4 elements in the array :
element - 0 : 6

element - 1 : 5

element - 2 : 8

element - 3 : 3

Sum of all elements stored in the array is : 22

(C) WAP in C to find reverse of array elements.

PROGRAM

```
#include <stdio.h>

#define MAX_SIZE 100 // Defines maximum size of array int main()
{
int arr[MAX_SIZE];
int size, i;
/* Input size of array */ printf("Enter size of the array: ");
scanf("%d", &size);
/* Input array elements */ printf("Enter elements in array: ");
for(i=0; i<size; i++)
{
scanf("%d", &arr[i]);
}
/*
*   Print array in reversed order
*/
printf("\nArray in reverse order: ");
for(i = size-1; i>=0; i--)

{
printf("%d\t", arr[i]);
```

```
}
return 0;
}
```

OUTPUT

```
$ gedit reverse1.c

$ gcc -o reverse1 reverse1.c

$ ./reverse1

Enter size of the array: 5 Enter elements in array: 70 45

87
32
09

Array in reverse order: 9  32       87       45       70
```

(D) WAP in C program to search an element in an array.

PROGRAM

```
arr=("apple" "banana" "mango" "cherry" "mango" "grape")
element="mango"
index=-1

for i in "${!arr[@]}";
do
   if [[ "${arr[$i]}" = "${element}" ]];
   then
      index=$i
      break
   fi
done

if [ $index -gt -1 ];
then
   echo "Index of Element in Array is : $index"
else
   echo "Element is not in Array."
fi
```

OUTPUT

Index of Element in Array is : 2

(E) WAP in C program to sort array elements in ascending or descending order.

PROGRAM

```
#!/bin/bash
echo "enter maximum number"
read n
# taking input from user
echo "enter  Numbers in array:"
for (( i = 0; i < $n; i++ ))
do
read nos[$i]
done
#printing the number before sorting
echo "  Numbers in an array are:"
for (( i = 0; i < $n; i++ ))
do
echo ${nos[$i]}
done
# Now do the Sorting of numbers
for (( i = 0; i < $n ; i++ ))
do
for (( j = $i; j < $n; j++ ))
do
if [ ${nos[$i]} -lt ${nos[$j]}  ]; then
t=${nos[$i]}
nos[$i]=${nos[$j]}
nos[$j]=$t
fi
done
done
# Printing the sorted number in descending order
echo -e "\nSorted Numbers "
for (( i=0; i < $n; i++ ))
do
echo ${nos[$i]}
done
```

OUTPUT

$ bash desc.sh

Enter maximum number

6

Enter Numbers in array

4 9 1 2 3 7

Numbers in an array are: 4 9 1 2 3 7

Viva Questions

1)How to find the current shell which you are using?

2)How to find all the available shells in your system?

3)How to read keyboard inputs in shell scripts?

4)How many fields are present in a crontab file and what does each field specify?

5)What are the two files of crontab command?

6)What command needs to be used to take the backup?

7)What are the different commands available to check the disk usage?

8)What are the different communication commands available in Unix/shell?

9)How to find out the total disk space used by a specific user, say for example username is John?

10)What is Shebang in a shell script?

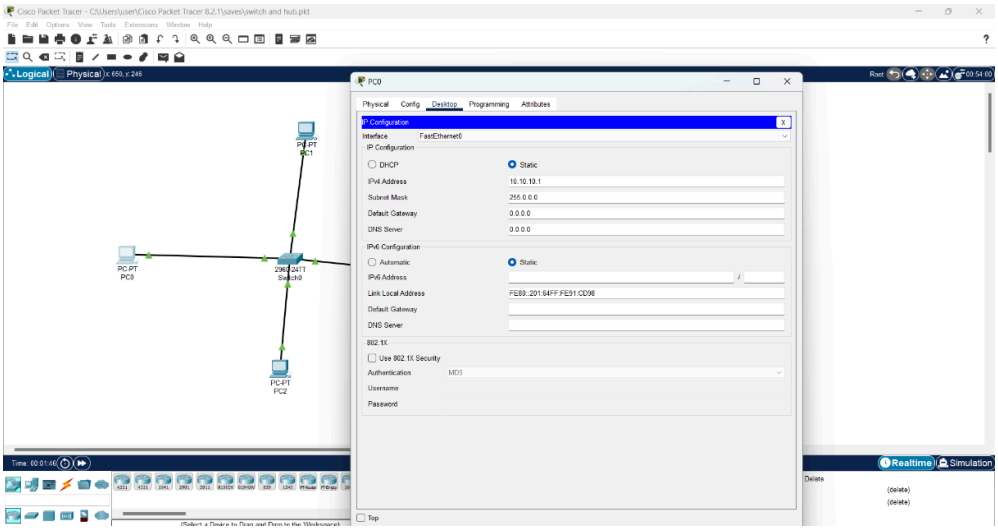
BEYOND THE SYLLABUS

EXPERIMENT-1

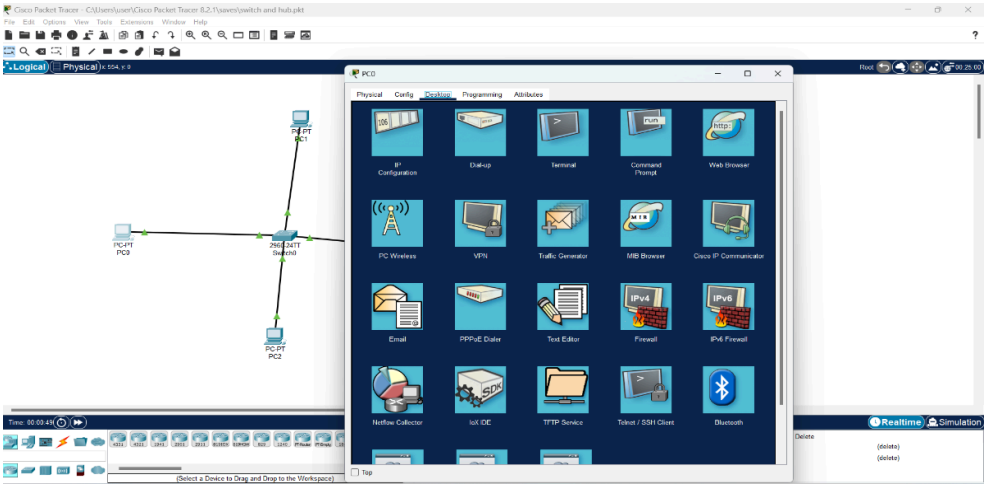
OBJECTIVE

Construct a network of hub and switch using Packet Tracer.

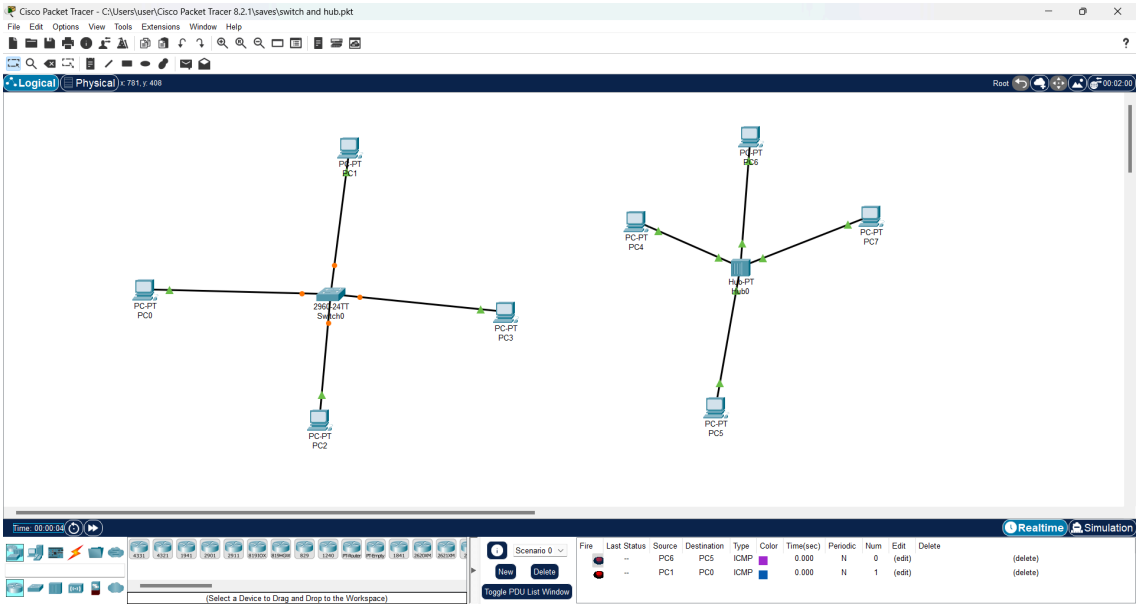
Step 1:



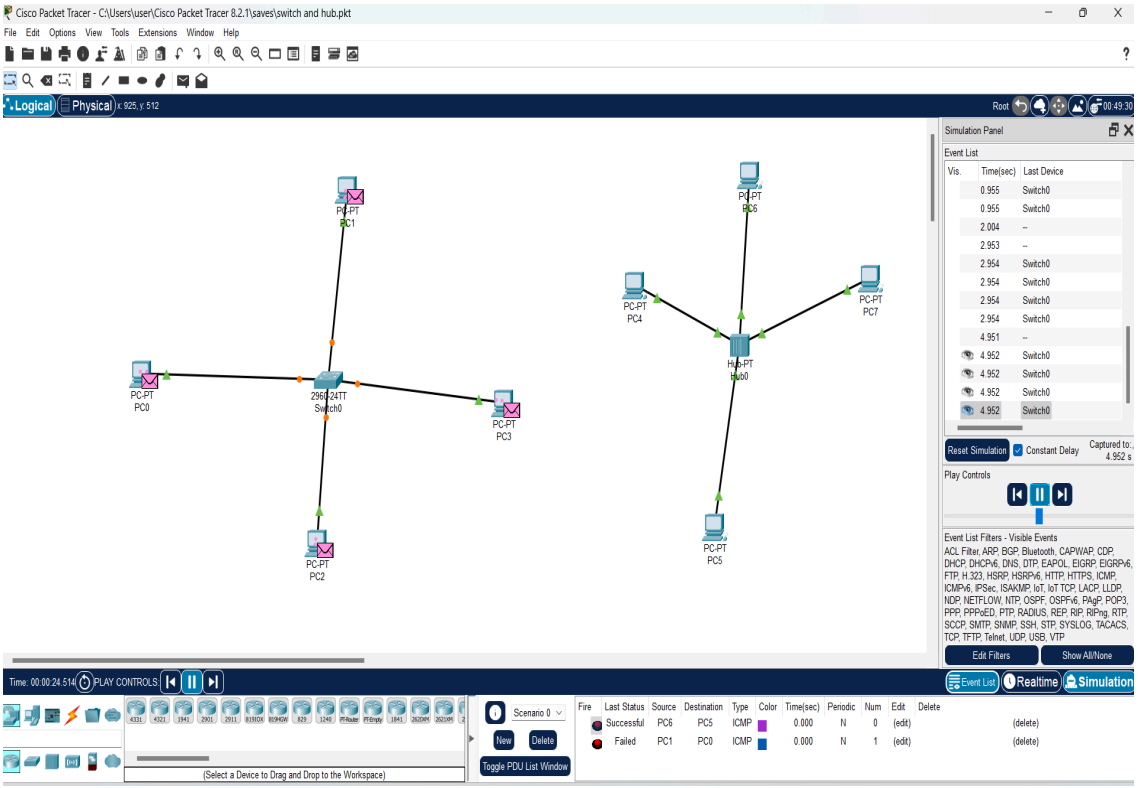
Step 2:



Step 3:


Cisco Packet Tracer - C:\Users\user\Cisco Packet Tracer 8.2.1\saves\switch and hub.pkt
File Edit Options View Tools Extensions Window Help
Logical Physical
PC-PT PC1
PC-PT PC0
2960-24TT Switch0
PC-PT PC3
PC-PT PC2
PC-PT PC6
PC-PT PC4
PC-PT PC7
Hub-PT Hub0
PC-PT PC5
Realtime Simulation
Scenario 0
New Delete
Toggle PDU List Window
(Select a Device to Drag and Drop to the Workspace)
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
-- PC6 PC5 ICMP 0.000 N 0 (edit) (delete)
-- PC1 PC0 ICMP 0.000 N 1 (edit) (delete)

Step 4:


Cisco Packet Tracer - C:\Users\user\Cisco Packet Tracer 8.2.1\saves\switch and hub.pkt
File Edit Options View Tools Extensions Window Help
Logical Physical
PC-PT PC1
PC-PT PC0
2960-24TT Switch0
PC-PT PC3
PC-PT PC2
PC-PT PC6
PC-PT PC4
PC-PT PC7
Hub-PT Hub0
PC-PT PC5
Simulation Panel
Event List
Vis. Time(sec) Last Device
0.955 Switch0
0.955 Switch0
2.004 --
2.953 --
2.954 Switch0
2.954 Switch0
2.954 Switch0
2.954 Switch0
4.951 --
4.952 Switch0
4.952 Switch0
4.952 Switch0
4.952 Switch0
Reset Simulation
Constant Delay
Play Controls
Event List Filters - Visible Events
ACL, Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, DTP, EAPOL, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IPSec, ISAKMP, IoT, IoT TCP, LACP, LLDP, NDP, NETFLOW, NTP, OSPF, OSPFv6, PAgP, POP3, PPP, PPPoED, PTP, RADIUS, REP, RIP, RIPng, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP, USB, VTP
Edit Filters
Show All/None
Event List
Realtime
Simulation
Scenario 0
New Delete
Toggle PDU List Window
(Select a Device to Drag and Drop to the Workspace)
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC6 PC5 ICMP 0.000 N 0 (edit) (delete)
Failed PC1 PC0 ICMP 0.000 N 1 (edit) (delete)

Viva Questions:

1. In which OSI layer hub is used ?
2. In which OSI layer switch is used ?
3. Why switch is more intelligent as compare with hub?
4. How we attach two systems in packet tracer?
5. How can you give IP address in packet tracer and after it how can you check connectivity between the systems.

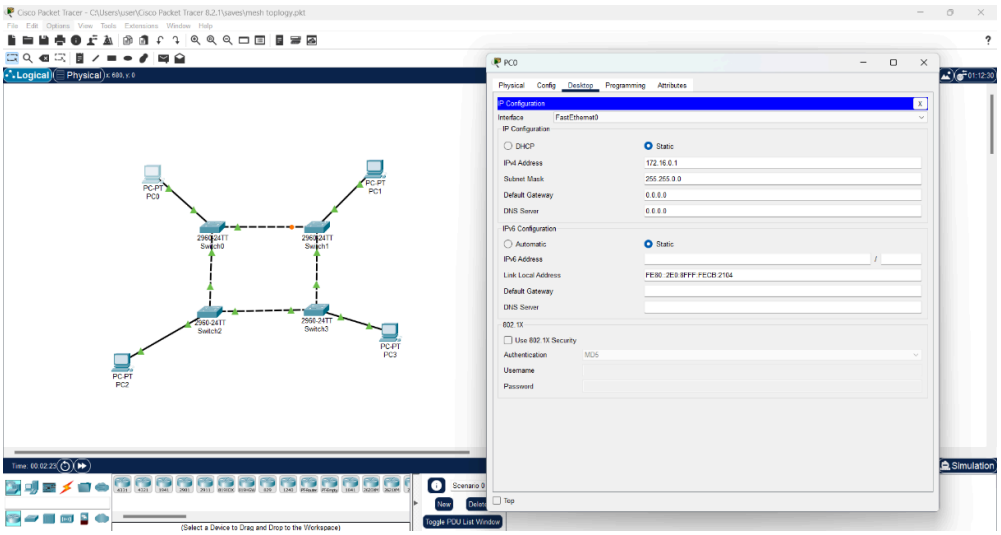
BEYOND THE SYLLABUS

EXPERIMENT-2

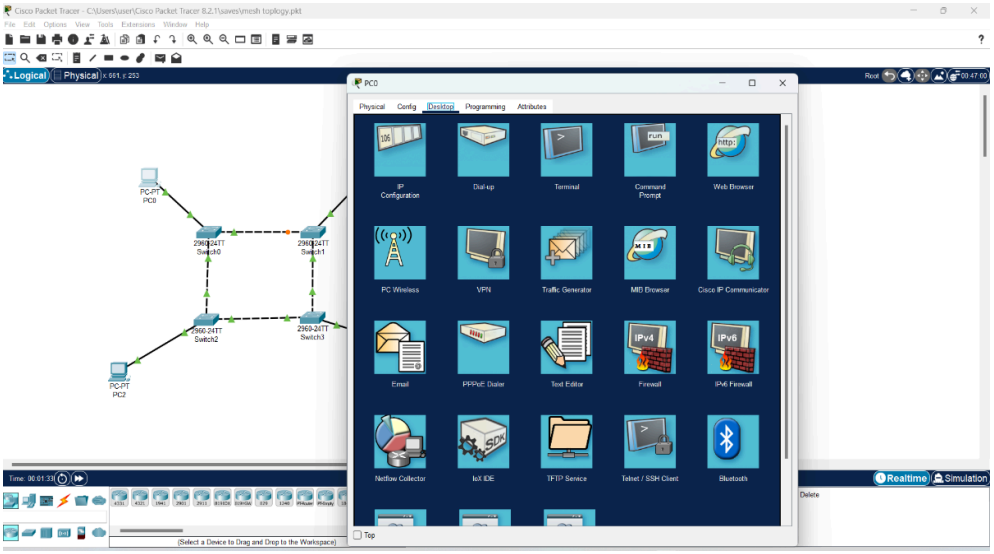
OBJECTIVE

Construct a network of Mesh topolgy using Packet Tracer.

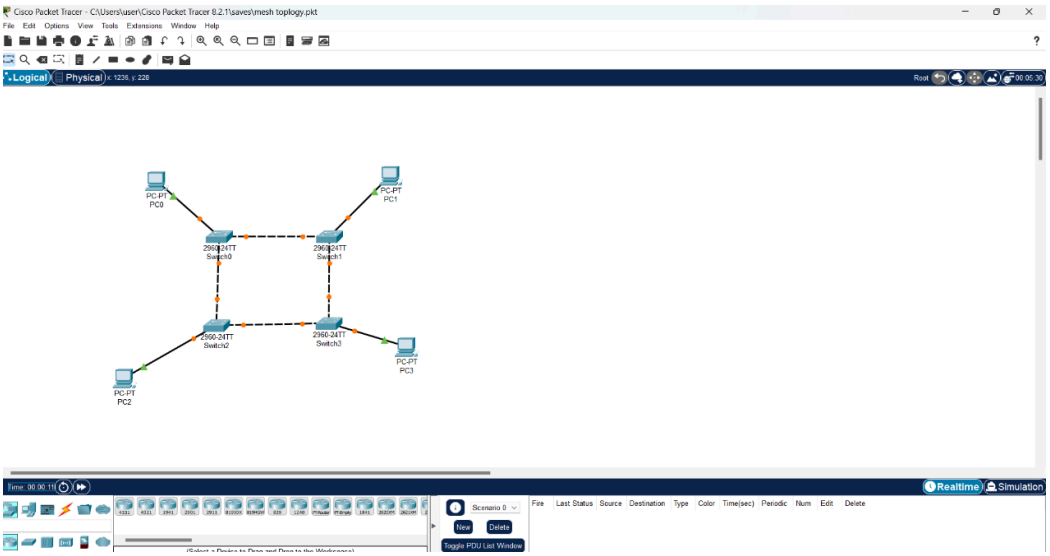
Step 1:



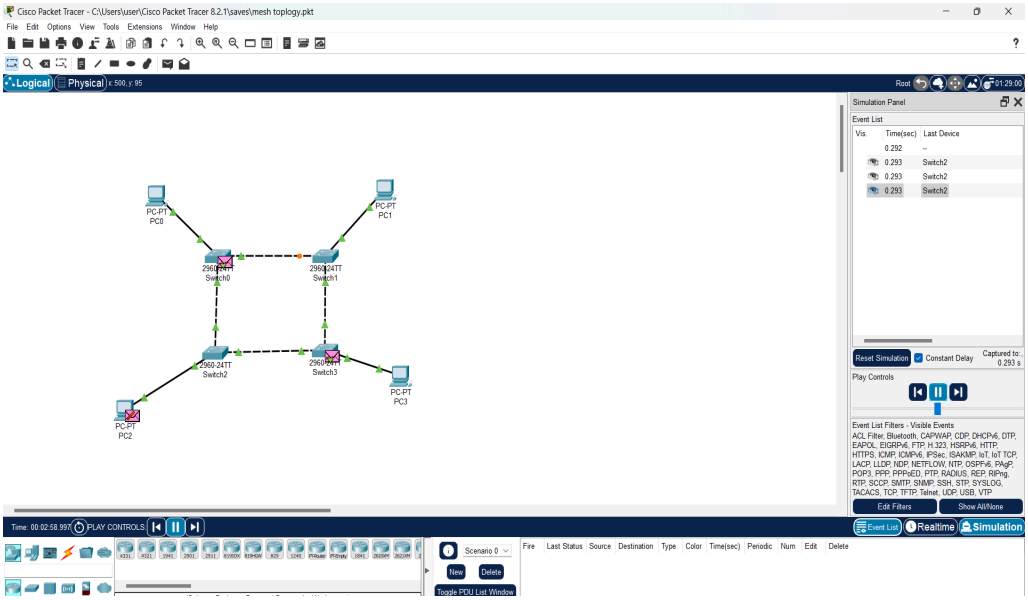
Step 2:



Step 3:


Cisco Packet Tracer - C:\Users\user\Cisco Packet Tracer 8.2.1\saves\mesh toplogy.pkt
Logical
Physical
PC-PT PC0
PC-PT PC1
2960-24TT Switch0
2960-24TT Switch1
2960-24TT Switch2
2960-24TT Switch3
PC-PT PC2
PC-PT PC3
Realtime
Simulation

Step 4:


Cisco Packet Tracer - C:\Users\user\Cisco Packet Tracer 8.2.1\saves\mesh toplogy.pkt
Logical
Physical
Simulation Panel
Event List
PC-PT PC0
PC-PT PC1
2960-24TT Switch0
2960-24TT Switch1
2960-24TT Switch2
2960-24TT Switch3
PC-PT PC2
PC-PT PC3
Realtime
Simulation

Viva Questions:

1. If there are m systems how many cables will required in mesh topology to connect them.
2. How mesh is different from star topology.
3. What are the advantages of mesh topology.
4. What are the steps to build a mesh network in packet tracer.
5. How can you check whether two systems are connected with each other or not.